

Biologically informed graph neural networks for perturba- tion effect prediction

– Integrating Multi-Modal Biological Data

Liuxi Mei

Supervisor : Oleg Sysoev
Examiner : Sebastian Mair

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

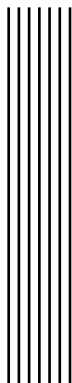
Abstract

Predicting gene expression responses after drug treatment is difficult because a drug can affect more than its main target, and the result also depends on the starting state of the cell. This thesis studies whether known biological information can improve this prediction task.

The proposed model is Counterfactual Attention GNN (CAGNN). It combines a matched control expression profile, drug target information, and a gene interaction graph from OmniPath. The prediction target is the response vector, defined as the difference between treatment and matched control expression. The model predicts both a mean response and a gene wise uncertainty value, which makes it possible to study how reliable the predictions are.

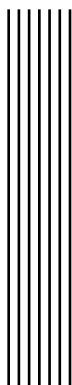
Experiments are performed on LINCS L1000 landmark genes under three test settings: warm, cold drug, and cold cell. CAGNN is compared with two baseline models, DeepCOP and GSNN, under the same split and pairing framework. CAGNN gives the most consistent strong results across the main metrics. It is best on sample wise PCC and top gene precision in all three splits, and it also gives the lowest MSE in warm and cold drug. In cold cell, DeepCOP gives a slightly lower MSE. Cold cell is the hardest setting on PCC and top gene precision.

The uncertainty analysis shows that higher predicted uncertainty is usually linked to larger prediction error, especially in harder cases. Attention case studies also show that highly weighted edges can be useful clues when predictions are accurate, but these weights should not be treated as direct biological proof.



Acknowledgments

I would like to express my sincere gratitude to my supervisor, colleagues, and family for their continuous support and encouragement throughout the course of this research.



Contents

Abstract	iii
Acknowledgments	iv
Contents	v
List of Figures	vii
List of Tables	x
1 Introduction	1
1.1 Background	1
1.2 Drug perturbation	2
1.3 High-throughput experiment	2
1.4 Prior knowledge of molecular interactions	4
1.5 Aim	4
1.6 Related works	5
2 Data	7
2.1 CMap LINCS Dataset	7
2.2 OmniPath Dataset	10
2.3 Data preprocessing	11
3 Theory	14
3.1 Deep neural networks	14
3.2 Graph neural networks	16
4 Methodology	25
4.1 Target	25
4.2 Inputs	25
4.3 Base model	28
4.4 GSNN baseline	30
4.5 Proposed model	31
4.6 Evaluation Metrics	34

5	Results	39
5.1	Evaluation framework	39
5.2	Error metric results	40
5.3	Precision metric results	42
5.4	Correlation metric results	43
5.5	Uncertainty analysis	45
5.6	Data efficiency evaluation	49
5.7	Interpretation and case studies	50
6	Discussion	56
6.1	Method	56
6.2	Result	56
6.3	Limitations and Future Work	57
6.4	Ethical Considerations	58
6.5	Usage of Generative AI	59
7	Conclusion	60
7.1	Research Questions	60
7.2	Conclusion	61
	Bibliography	62

List of Figures

1.1	Conceptual illustration of the AstraZeneca “compound 12” case. Although the compound targets KRAS, additional off-target effects may spread through molecular networks and cause unwanted outcomes.	2
1.2	Sketch of the high-throughput well plate experiment, adapted from the LINCS L1000 assay description [1]. A well is a small, separate container that holds cells for individual testing.	3
1.3	Example of a 96 well plate layout used in high-throughput screening. Each row tests one compound across a range of doses through serial dilution. DMSO only wells serve as controls without the active compound.	3
1.4	Left: Example of a real gene network, showing complex connections. Right: A simple drawing of a graph neural network (GNN), where genes and proteins are nodes, and their interactions are edges.	5
2.1	LINCS L1000 data processing workflow. The pipeline converts raw fluorescence intensities (Level 1) to deconvoluted probe values (Level 2), normalized and inferred gene expression (Level 3), differential expression signatures (Level 4), and consensus signatures (Level 5).	8
2.2	Schematic overview of the OmniPath interaction tables and the graph construction used in this thesis. Each row in OmniPath describes a directed regulator to target relationship together with sign related fields (stimulation or inhibition) and consensus annotations. We filter to retain directed regulatory relationships with strong consensus and then convert the resulting edge list into a signed gene graph for downstream message passing. . .	11
2.3	Effect of successive filtering steps on the number of treatment profiles retained for modelling in a single dose and single time point setting. The final step counts treatment profiles that can be paired with at least one matched vehicle control under the context matching rules.	12
3.1	A Feed-Forward Network with several hidden layers. When an FFN contains several hidden layers, it can also be viewed as a deep neural network.	14
3.2	Dropout during training. Some hidden units are removed, so the model uses different paths in different updates.	15
3.3	Main components of a graph. A graph can contain node attributes, edge attributes, and global information.	16
3.4	From a graph to its adjacency matrix. The graph structure is converted into a matrix form so that graph connections can be used in computation.	17

3.5	Message passing in a single layer. Messages from neighbors are grouped together and then used to update the target node.	18
3.6	How message passing expands across layers. The yellow node N_1 is the target node. After one layer, it receives information from its one hop neighbors N_2 , N_3 , and N_4 . After two layers, information from the two hop neighbors N_5 to N_9 can also reach N_1 through the intermediate nodes.	18
3.7	Three common GNN output levels. A GNN can produce one output for each node, one output for a selected edge, or one output for the whole graph after a readout step.	19
3.8	A simple view of Graph Convolutional Networks. The input graph is processed by graph convolution layers, and the learned node representations are used for prediction.	20
3.9	Computation of attention scores and normalized attention weights in a graph attention layer. Node features are projected, compared, passed through LeakyReLU, and then normalized by softmax.	21
3.10	A residual connection. The input follows both the main stacked layers and a shortcut path, and the two outputs are added together.	22
3.11	A Position-wise Feed-Forward Network block. It consists of two linear transformations with a nonlinear activation function in between.	23
4.1	Control input and its node wise projection. Each sample is represented as a 978 dimensional gene expression vector aligned to the gene order. The same shared projection layer maps each gene wise scalar input to a 64 dimensional node feature.	26
4.2	Drug target input and its node wise projection. Each sample is represented as a binary target vector aligned to the gene order. After adding one extra feature dimension, the same shared projection layer maps each gene wise scalar input to a 64 dimensional node feature.	26
4.3	Cell identity input and its embedding. Each cell line name is first mapped to an integer index by a label encoder, and then a learnable embedding layer maps the index to a 64 dimensional cell representation. The same cell embedding is shared by all samples from the same cell line.	27
4.4	Control derived cell context. The control profile is normalized and encoded into a sample specific context vector. This context summarizes the baseline expression state of the sample before drug perturbation.	28
4.5	Structure of the baseline model. Control gene expression and drug features are concatenated and passed through a feed forward network to predict a mean response and a log variance for each gene.	29
4.6	Structure of the proposed model. The model combines the biological interaction graph, the control profile, and drug target information to predict the gene wise response.	31
4.7	Flow of one graph attention block used in the proposed model. The input node features are linearly projected, standard graph attention scores are computed and normalized with softmax, and the multi head output is followed by dropout, residual addition, layer normalization, and a position wise feed forward network.	32
4.8	Illustration of how mean test PCC is computed for the <code>Mean</code> baseline. An average response vector is first estimated from the training set and then repeated as the prediction for every test sample. PCC is computed between each predicted row and the corresponding true response row using Eq. 4.19.	35
4.9	Illustration of how mean test PCC is computed for the <code>mean_cell</code> baseline. The training responses are first grouped by cell line, and one average response vector is computed for each cell. Each test sample then receives the average vector of its corresponding cell line. Sample wise PCC is computed between each predicted row and the corresponding true response row using Eq. 4.19, and the final test PCC is the mean of these sample wise PCC values.	36
5.1	Simple view of the evaluation configuration used in this chapter.	40

5.2	Test MSE in warm, cold drug, and cold cell for the learned models and the simple mean baselines. <code>mean_drug</code> is not shown in cold drug and <code>mean_cell</code> is not shown in cold cell because these baselines cannot be used there.	41
5.3	Gene level scatter plots for DeepCOP, GSNN, and CAGNN in the three splits.	42
5.4	The left panel shows Pos P@20 and the right panel shows Neg P@20 for DeepCOP, GSNN, and CAGNN in the three splits.	43
5.5	Test PCC in warm, cold drug, and cold cell for the learned models and the simple mean baselines. <code>mean_drug</code> is not shown in cold drug and <code>mean_cell</code> is not shown in cold cell because these baselines cannot be used there.	44
5.6	Gene level distribution in the three splits. The figure has three rows, one for each split. Each row shows x_{deg} for twelve genes with strong true response. Ground truth, DeepCOP, GSNN, and CAGNN are shown for each gene.	45
5.7	Uncertainty summary in warm, cold drug, and cold cell, based on the test set. The left panel shows sample level uncertainty in the three splits. The middle panels compare uncertainty with MSE and PCC. The right panel shows MSE and PCC for low, medium, and high uncertainty groups in each split.	47
5.8	The left panel shows the top 20 genes with the highest mean predicted standard deviation on the test set. The right panel compares mean gene level σ with gene level MSE for all target genes on the test set.	48
5.9	Case studies with uncertainty bands in warm, cold drug, and cold cell. In each split, the left panel shows a lower uncertainty sample with a better fit, and the right panel shows a higher uncertainty sample with a larger error. Each panel uses the twelve genes with the largest absolute true response in that sample. The plots show the true response, the predicted mean, and the prediction band $\mu \pm 2\sigma$	49
5.10	Warm split data efficiency results. The left panel shows test PCC and the right panel shows test MSE for DeepCOP, GSNN, and CAGNN across the displayed training budgets. In the updated DeepCOP budget runs, DeepCOP also improves clearly as more training data are used, while GSNN changes less across the same budgets.	50
5.11	Attention heatmap for the top 20 best drugs in the cold drug split. For each drug, the heatmap shows the strongest edges between different genes. The values are scaled within each drug. Darker blue regions show where the model puts more attention.	51
5.12	Attention heatmap for the top 20 worst drugs in the cold drug split. For each drug, the heatmap shows the strongest edges between different genes. The values are adjusted within each drug. Darker blue regions show where the model puts more attention.	52
5.13	Attention heatmap for the top 20 best drugs in the cold cell split. For each drug, the heatmap shows the strongest edges between different genes. The values are scaled within each drug. Darker blue regions show where the model puts more attention.	53

List of Tables

2.1	Main LINCS fields used in this chapter. The table gives a short meaning for each field and shows which table it comes from.	9
4.1	Main configuration of the baseline model.	30
4.2	Training configuration of the proposed model.	34
5.1	Summary of the evaluation protocol used in the thesis.	39
5.2	Test MSE under the three evaluation splits.	40
5.3	Top gene precision results computed from the exported prediction files.	43
5.4	Test PCC under the three evaluation splits.	43
5.5	Regulation sign accuracy under the three evaluation splits.	44
5.6	Comparison of cell information variants in CAGNN. The table reports test MSE and test PCC for cell identity and cell context, together with their differences. Here Δ means cell context minus cell identity, so negative Δ MSE and positive Δ PCC indicate improvement.	45
5.7	Coverage of the prediction band $\mu \pm 2\sigma$ on the full test set.	47
5.8	Short explanations of selected biology terms used in the attention analysis and case studies.	50
5.9	Top high attention edges for the best and worst cold drug examples. The left three columns show the best cold drug, BRD-K86797399. The right three columns show the worst cold drug, BRD-A72066420. The type column indicates whether the prior graph marks the edge as stimulation or inhibition. Attention values come from the last attention layer.	53
5.10	Top high attention edges for one shared drug in warm and cold cell. The same drug, BRD-K87909389, is shown in both settings. The type column indicates whether the prior graph marks the edge as stimulation or inhibition. Attention values come from the last attention layer.	54
5.11	Top high attention edges for a second shared drug in warm and cold cell. The drug is BRD-K56301217. Its known target genes are BCL2, BCL2L1, and BCL2L2. The type column indicates whether the prior graph marks the edge as stimulation or inhibition. Attention values come from the last attention layer.	55



1 Introduction

1.1 Background

Drug discovery is slow, expensive, and uncertain. Many candidate drugs fail before they reach the market [2]. One reason is that drug effects are hard to predict. A drug may bind to one target protein, but the final response depends on many connected genes, proteins, and signaling pathways inside the cell [3, 4]. Because these systems are highly connected, it is difficult to predict how a drug changes the state of a cell.

The AstraZeneca “compound 12” case, reported in 2025, shows this problem clearly. Figure 1.1 gives a simple illustration. The compound was designed to act on KRAS, but later analysis suggested that it also affected other targets and caused toxic effects in animal studies [5]. The case shows an important point. A drug may reach its intended target and still lead to an unwanted outcome. Its effect can spread through a larger molecular system. This thesis studies that broader prediction problem. The goal is to predict how gene expression changes after drug treatment, relative to a matched control, in a specific cell type.

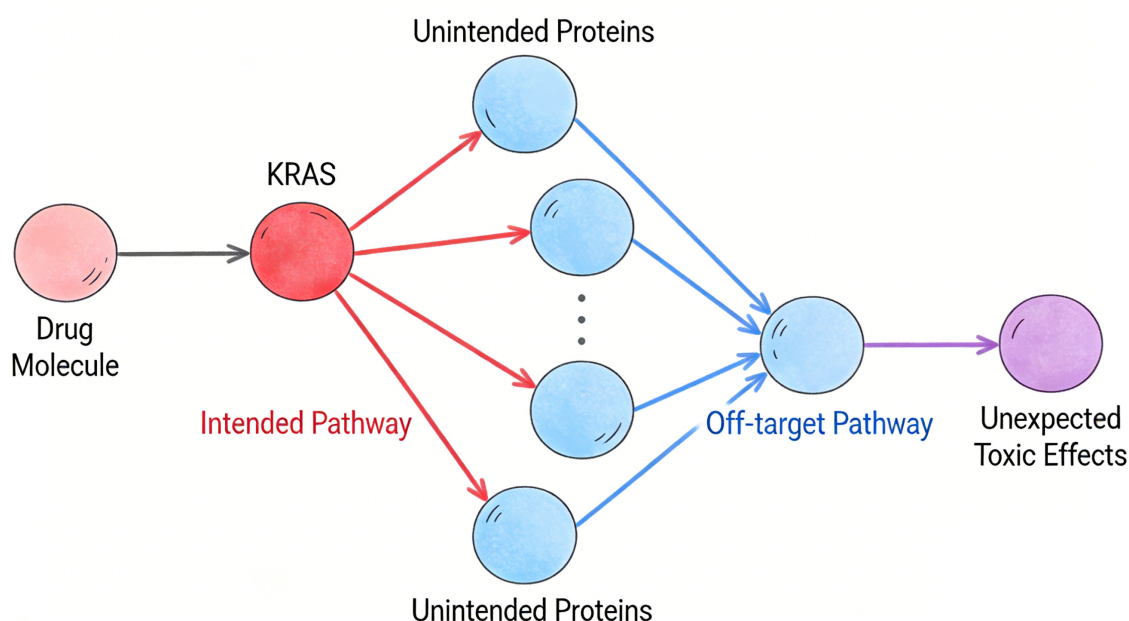


Figure 1.1: Conceptual illustration of the AstraZeneca “compound 12” case. Although the compound targets KRAS, additional off-target effects may spread through molecular networks and cause unwanted outcomes.

1.2 Drug perturbation

Drug perturbation means using a drug to change the state of a cell so that the response can be studied. A drug usually starts by binding to one or more target proteins. This event may block a protein, activate it, or change its function in another way. The effect usually does not stop there. The signal can spread through interactions inside the cell and affect many downstream genes.

In this work, the cellular response is described through gene expression. Gene expression shows how active each gene is. After drug treatment, some genes become more active and others become less active. Predicting these changes gives a broad summary of the molecular response and can be measured in large datasets.

1.3 High-throughput experiment

Researchers use high-throughput experiments to study drug perturbations at scale. The idea is simple. Many cell lines are grown in multi-well plates. Many drugs are applied under controlled conditions. Gene expression is measured after treatment. This design makes it possible to compare many perturbations in a standard way. Figure 1.2 shows this basic idea in a simple form.

This thesis uses the LINCS L1000 dataset. LINCS records gene expression responses for many compounds, cell lines, doses, and treatment times. Each well contains cells under one experimental condition. After treatment, the assay measures 978 landmark genes. The expression of additional genes can then be inferred from these measurements.

Each plate includes two main types of wells:

1. **Treatment wells:** These contain cells exposed to a specific drug, usually dissolved in DMSO.
2. **Control wells:** These contain cells exposed only to DMSO [6], without the drug. They provide a reference for the same experimental setting without the drug effect.

The key comparison is the difference between a treatment well and a matched control well. This comparison helps separate the drug effect from the background state of the cell. The control must

come from a similar treatment setting. The dataset therefore records the cell line, drug dose, treatment time, and plate information, so treatment and control wells can be matched under comparable conditions. These variables matter because the same drug can act differently in different cell types and under different experimental settings.



Figure 1.2: Sketch of the high-throughput well plate experiment, adapted from the LINCS L1000 assay description [1]. A well is a small, separate container that holds cells for individual testing.

Figure 1.3 gives a more concrete example of how such a plate can be organized in practice. Each row can be assigned to one compound. Several wells in the same row form a dilution series, so the same compound is tested at different doses. DMSO only wells are used as controls. In this way, one plate can contain many related test conditions while still keeping a clear reference for comparison.

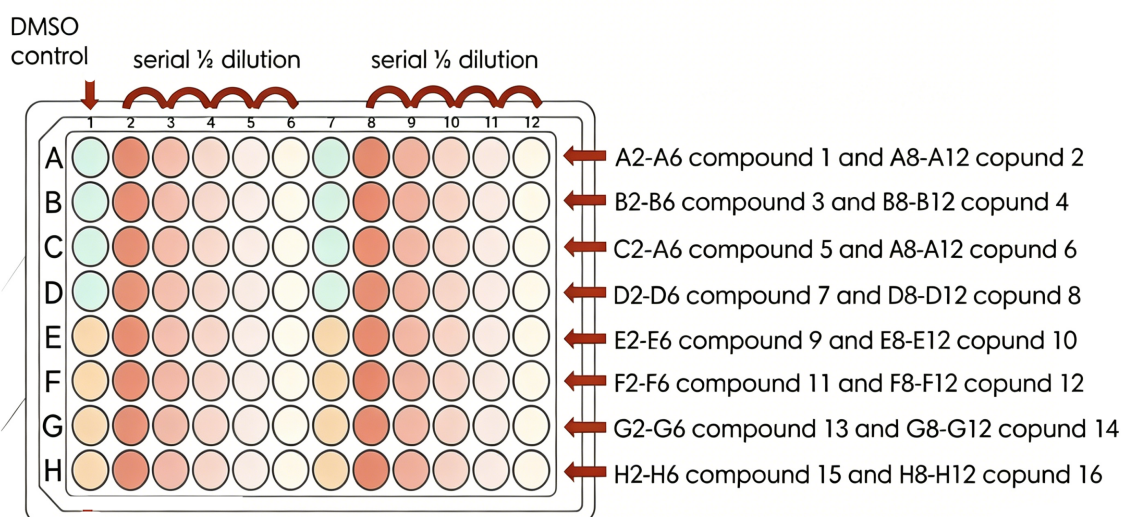


Figure 1.3: Example of a 96 well plate layout used in high-throughput screening. Each row tests one compound across a range of doses through serial dilution. DMSO only wells serve as controls without the active compound.

1.4 Prior knowledge of molecular interactions

Another source of information comes from biological knowledge databases. These resources summarize known links among genes and proteins, such as control and signaling relations. This information is useful because drug effects do not come from one target alone. A drug may start from a small set of targets, but the response is shaped by how signals move through the surrounding molecular network.

OmniPath is one example of such a database. It combines molecular interaction information from many sources and provides directed relationships that can be used to build a graph representation of the cell [7]. In this thesis, the OmniPath network is matched to the genes measured in LINCS L1000, so known biology and experimental data use the same gene space.

This thesis combines these sources of information. Drug treatment data show what changes after treatment. The interaction network gives information about how those changes may spread through the cell. Drug target information marks where the effect begins. Together, they can help the model predict better than by using expression data alone.

1.5 Aim

This thesis studies whether a graph based model can predict gene expression changes after drug treatment from the baseline state of the cell, known drug targets, and a molecular interaction network. More accurate response prediction can help researchers test compounds, choose which experiments to perform next, and understand drug action more clearly.

The aims of this thesis are:

1. To build a graph neural network (GNN) based on the OmniPath molecular interaction network. This model predicts drug related gene expression changes by combining baseline cell data and drug target information.
2. To test how well the model works on unseen drugs and unseen cell lines (where test drugs or cells were never seen during training).
3. To compare this model with baseline models that do not use a gene network, and to examine the contribution of the drug target information.

The proposed model uses a graph attention network (GAT) [8] on the molecular interaction graph. In this graph, nodes correspond to the measured genes and edges represent prior molecular interactions.

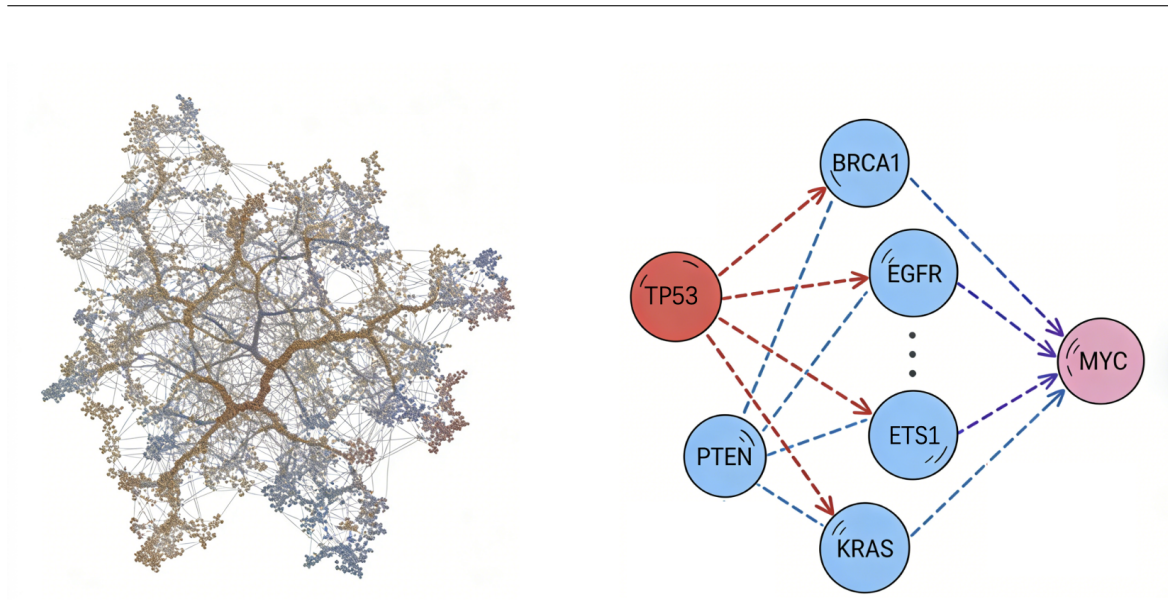


Figure 1.4: Left: Example of a real gene network, showing complex connections. Right: A simple drawing of a graph neural network (GNN), where genes and proteins are nodes, and their interactions are edges.

Figure 1.4 gives a simple view of this modeling idea. The molecular network provides the graph structure, and the GNN passes information from drug targets to downstream genes.

1.6 Related works

Graph neural networks are now widely used in biology because many biological systems can be represented as graphs. A recent review summarizes GNN applications in single cell omics and discusses common architectures such as GCN, GraphSAGE, and GAT [9, 10, 11, 12, 8]. These models are useful because each node can update its representation by using both its own features and information from connected neighbours. In biology, this makes them a natural choice for pathways, control links, and other structured molecular systems.

More recent graph models also use Transformer style block designs. Graphormer shows that a standard Transformer architecture, combined with graph specific structural encodings, can work well for graph representation learning [13]. GraphGPS extends this direction with a modular graph Transformer design that combines local message passing with global attention and reports strong performance across many graph tasks [14]. These studies are important here because they support the broader idea that attention, FFN, residual connections, and normalization can also form useful repeated blocks in graph models.

Several studies have shown the value of graph based learning in perturbation biology. GEARS uses a gene relationship graph to predict responses to genetic perturbations in single cell data [15]. Cellograph models multi condition single cell datasets with graph based learning [16]. Evans et al. also study graph structured neural networks for perturbation biology more broadly [17]. These studies focus on different data types and tasks, but they support the same basic idea that graph structure can help connect an intervention to downstream molecular changes.

Another line of work predicts drug effects without an explicit biological interaction graph. Large resources such as LINCS L1000 and Connectivity Map make this possible because they provide gene expression responses for many compounds and cell lines [1]. DeepCOP [18] is one example. It uses a multilayer perceptron and drug information to predict gene expression changes. This type of model is simple and scalable, and it provides a strong baseline. Still, it treats genes as separate outputs and does not use known molecular links among genes and proteins.

A different line of work uses known biology more directly. Regulation Flow Analysis models how drug effect signals may move through an interaction network collected from earlier studies and uses this structure to suggest possible mechanisms [19]. This kind of method is attractive because it is easier to explain biologically. Its limitation is that the signal moving rule is largely fixed by the network and does not learn directly from large drug response datasets. By contrast, data driven graph neural networks can learn how strongly information should move while still using the network structure. Some recent studies combine graph structure with supervised prediction for drug response related tasks [20, 21]. These studies suggest that molecular networks can give a useful starting structure when labels are noisy or when the response follows many indirect molecular steps. They also show that graph quality matters. Resources such as OmniPath [7] are useful because they provide directed interactions and sign information for signal flow.

Evaluation is also important in this area. Many studies report performance under random splits, where related drugs or cell contexts may appear in both training and test sets. For drug response prediction, this can make the model look stronger than it really is on unseen cases. A stricter setting tests unseen drugs or unseen cell lines. This thesis follows that view and studies whether a graph based model can work beyond the conditions seen during training.



2 Data

This thesis uses two data sources. The first source is LINCS L1000 gene expression data. Another source is the OmniPath interaction database. These two sources give different kinds of information. LINCS shows how cells respond to drug treatment. OmniPath gives known information about how genes and proteins are linked. LINCS provides the drug response data, while OmniPath provides the graph used by the graph based models.

The data can be grouped into three connected parts. One part is the gene expression matrix, which stores treatment and control profiles. Another part is the metadata and compound annotation tables, which store cell line, dose, time, plate information, and drug target annotations. The last part is the OmniPath interaction network. The sections below explain these parts and how they are linked by shared identifiers.

2.1 CMap LINCS Dataset

The NIH Common Fund's Library of Integrated Network based Cellular Signatures (LINCS) provides large datasets for studying how cells respond to chemical and genetic changes. A major part of LINCS is the L1000 assay. This assay directly measures the expression of 978 landmark genes [1]. These genes were chosen because they capture a large part of the change in gene expression across many biological conditions. The L1000 pipeline can also estimate additional genes from the measured landmark genes, which gives a wider gene space of 12,328 genes in total. The data are widely used because they cover many compounds, cell lines, and experimental conditions, together with matched control samples.

L1000 Data Processing Flow

LINCS L1000 data have been released through the GEO archive and later combined resources such as the CMap LINCS 2020 release. In this releases, the data are commonly organized into several data levels. These levels represent different stages of processing for the same experiments:

1. **Level 1: Raw fluorescence intensities.** These are the direct assay measurements before correction.
2. **Level 2: Deconvoluted probe values.** Probe signals are converted to gene level values.
3. **Level 3: Normalized and inferred gene expression.** Data are normalized across samples and batches. Non landmark genes are estimated from the landmark genes.

4. **Level 4: Differential expression signatures.** Treatment and control profiles are compared to show drug effects.
5. **Level 5: Consensus signatures.** Replicates and conditions are combined into more stable summary signatures.

This thesis uses Level 3 data. Figure 2.1 shows the full processing flow. These data provide normalized and inferred gene expression profiles in a common gene space across samples.

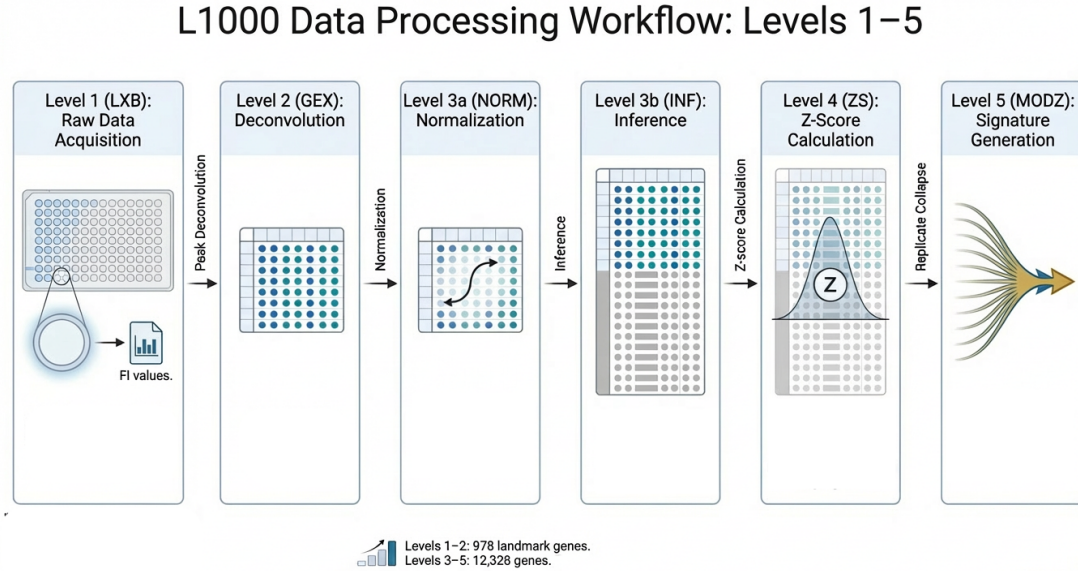


Figure 2.1: LINCS L1000 data processing workflow. The pipeline converts raw fluorescence intensities (Level 1) to deconvoluted probe values (Level 2), normalized and inferred gene expression (Level 3), differential expression signatures (Level 4), and consensus signatures (Level 5).

Each Level 3 experiment is represented as a vector of gene expression values in a shared gene space. We denote a profile by $x \in \mathbb{R}^G$. Here $G = 12,328$ for the full gene set and $G = 978$ when only landmark genes are used. Profiles are indexed by identifiers such as `distil_id`. They are stored in large matrices where rows are profiles and columns are genes in a fixed order. A separate gene information table maps columns to gene identifiers.

Treatment and control data

LINCS experiments include both compound treatment profiles and vehicle control profiles. These profiles cover many cell lines, compounds, and experimental conditions. Each profile is stored as a gene expression vector in the shared gene space. The related metadata, such as cell line, compound, dose, and time, are stored in a separate table and linked to profiles through `distil_id`. Treatment profiles describe the cell state after drug exposure. Control profiles describe the same type of setting without the drug. The difference between treatment and control is the basis of the perturbation response used later in the thesis.

Metadata

Metadata come from the LINCS signature annotation table that is released with the Level 3 expression data [1]. Each row describes one signature. The table includes compound identity, cell line, experimental conditions, quality control, and batch related variables. These fields give the biological

and technical context for the expression matrices. They also link expression profiles to compound annotations. Reliable pairing, filtering, grouping, and evaluation depend on this information. To make these fields easier to read, Table 2.1 lists the main identifiers and quality related variables that appear in this chapter. They belong to four simple groups: sample identifiers, experimental condition fields, technical batch fields, and quality control fields.

Table 2.1: Main LINCS fields used in this chapter. The table gives a short meaning for each field and shows which table it comes from.

Field	Meaning	Source table
<code>distil_id</code>	One expression profile identifier. It points to one row in the Level 3 expression matrix.	Level 3 expression matrix
<code>distil_ids</code>	One or more <code>distil_id</code> values linked to the same signature in the metadata table.	meta table
<code>sig_id</code>	Unique identifier for one perturbation signature.	meta table
<code>pert_id</code>	Compound identifier used to link signatures to compound annotations.	meta table, compound annotation table
<code>bead_batch</code>	Identifier for the technical bead batch used in the assay.	meta table
<code>det_plates</code>	Identifier for the detection plate used in the measurement process.	meta table
<code>is_hiq</code>	Simple high quality flag for a signature.	meta table
<code>qc_pass</code>	Flag that shows whether a signature passed quality control.	meta table
<code>tas</code>	Summary score that reflects signature strength and reproducibility.	meta table

Each signature is linked to a compound identifier `pert_id` and a cell line name `cell_iname`. Each signature also has a unique signature identifier `sig_id`. The field `distil_ids` contains one or more expression profile identifiers, `distil_id`, that point to the corresponding rows in the Level 3 matrices.

The metadata also record the experimental condition through fields such as `pert_dose`, `pert_dose_unit`, `pert_time`, and `pert_time_unit`. In addition, they include technical context variables such as `bead_batch`, `det_plates`, and `det_wells`. These fields are useful because two samples may look different for technical reasons even when their biology is similar. Without this information, treatment and control samples could be matched across different technical conditions. Quality control is also described in the metadata [1]. Some signatures are weak or noisy, so the table includes fields such as `is_hiq` and `qc_pass`. Other summary fields, such as `tas`, describe how strong and reproducible a signature is. These variables help identify more reliable profiles and remove noisier ones.

Drug target

Drug related information is collected from the compound annotation table. This table is indexed by `pert_id`. It stores compound names in `cmap_name`, target annotations in `target`, and mechanism of action descriptions in `moa`. In this thesis, it is used to identify which genes are reported as targets of each drug, so that drug specific input signals can be matched to the expression matrix and the OmniPath graph.

The `target` field lists reported protein or gene targets for a subset of compounds. One drug may have several targets, so the field can contain more than one gene. During preprocessing, these target names are mapped to the gene identifiers used in the selected expression space. Only targets that align with the expression matrix and the graph can be used by the model. Target information is not available for all compounds in LINCS because many compounds are still not fully characterized.

Table association

The main tables are linked by shared identifiers. This makes it possible to trace each expression profile back to its experimental context, including cell line, dose, time, batch, and plate information, and to its compound annotation. These links support consistent filtering, grouping, and matched treatment control comparisons.

Level 3 expression matrices are indexed by `distil_id`. The metadata table contains `distil_ids`, which stores one or more `distil_id` values for each signature. This field links metadata to expression profiles.

Each signature row in the metadata table also includes a compound identifier `pert_id`. The compound annotation table uses the same identifier as its primary key, which links signatures to drug target annotations.

The metadata further include biological and technical context variables such as `cell_iname`, `bead_batch`, and `det_plates`. These fields support grouping and stratified analysis by biological and technical context.

2.2 OmniPath Dataset

OmniPath is a public database of gene and protein interactions in human cells. It collects control related information from many studies and databases. This thesis uses the regulatory subset of OmniPath. Each row describes a regulator protein, its target gene, and whether the reported effect is activating or inhibiting.

For each interaction, OmniPath stores fields that describe direction, sign, and evidence. For example, `is_directed`, `is_stimulation`, and `is_inhibition` indicate whether the interaction is directed and whether it is activating or inhibiting. OmniPath also stores evidence related fields such as `sources`, `n_sources`, `n_primary_sources`, and `n_references`.

OmniPath combines interactions from many sources, so different sources may report different labels for the same edge. One source may report stimulation, while another may report inhibition. OmniPath stores both individual labels and combined fields. These fields can be used during graph construction.

OmniPath also provides evidence counts for each edge, such as `n_sources`, `n_primary_sources`, and `n_references`. These fields summarize how many databases and publications support an interaction. In this thesis, the combined labels and evidence counts are used to filter edges and keep interactions with stronger support, although the final graph can still contain imperfect edges.

Genes from the LINCS expression matrix are mapped to nodes in the OmniPath regulon graph. The resulting directed network encodes transcription factor to gene relationships that are supported by published studies. When this network is combined with gene expression data, the model can use both measured responses and known control information. Figure 2.2 illustrates the table structure used for graph construction.

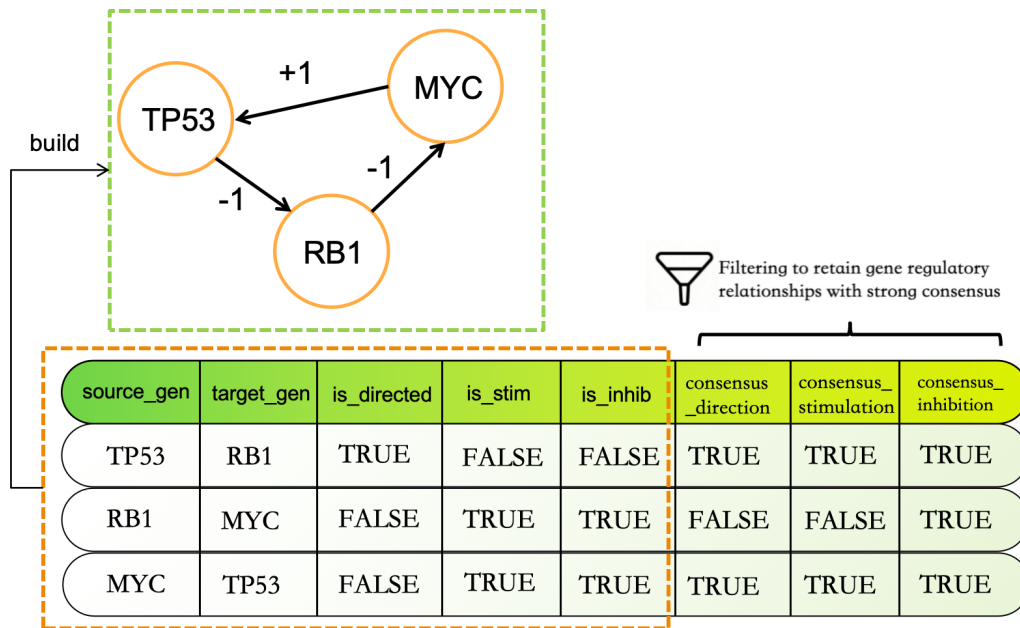


Figure 2.2: Schematic overview of the OmniPath interaction tables and the graph construction used in this thesis. Each row in OmniPath describes a directed regulator to target relationship together with sign related fields (stimulation or inhibition) and consensus annotations. We filter to retain directed regulatory relationships with strong consensus and then convert the resulting edge list into a signed gene graph for downstream message passing.

2.3 Data preprocessing

The LINCS Level 3 matrices and metadata are converted into the matrices and vectors used by the models. The main steps are:

1. select one gene set;
2. build matched treatment and control candidates;
3. compute treatment response targets;
4. prepare drug representations for model input.

CMap LINCS L1000 preprocessing

Gene set selection

Two gene spaces are supported. In landmark mode, the 978 landmark genes are used in the order given by the landmark gene list. In full gene mode, all 12,328 genes are loaded from the LINCS gene information table and `pr_gene_id` is used as the identifier. In both cases, one fixed gene order is used across control profiles, treatment profiles, response targets, and drug vectors. This keeps all matrices aligned.

Metadata filtering

Preprocessing starts from the metadata table, which links each profile to its cell line, compound identifier, perturbation dose, perturbation time, and batch information. Figure 2.3 summarizes the effect of the main filtering steps. The pipeline keeps two profile types, compound treatments `trt_cp` and vehicle controls `ctl_vehicle`. To reduce heterogeneity across experiments, one time point and

one dose setting are used. By default, profiles measured at 24 hours and treatments with dose close to 10 μM are kept. High quality treatment profiles are also retained, for example by using `is_hiq = 1`. Optional filtering by cell line is supported through `siginfo_beta.txt`.

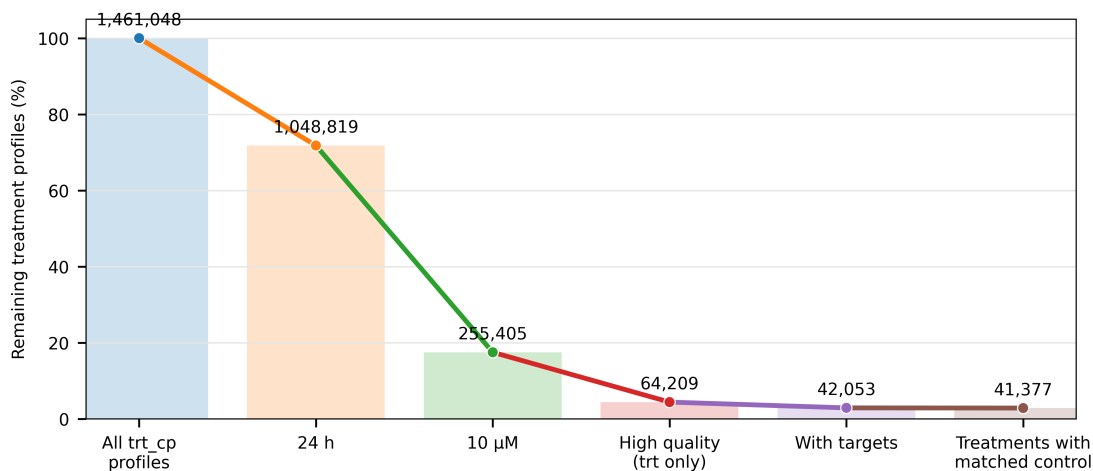


Figure 2.3: Effect of successive filtering steps on the number of treatment profiles retained for modelling in a single dose and single time point setting. The final step counts treatment profiles that can be paired with at least one matched vehicle control under the context matching rules.

From the filtered metadata, control profiles are collected and the `distil_ids` field is expanded so that each profile has one unique identifier. For each control profile, the cell line, bead batch, and detection plate are recorded. The expression vectors for these identifiers are then loaded from the Level 3 matrix. The loader reads only the requested identifiers and only the selected gene columns. The same procedure is applied to treatment profiles. The field `distil_ids` is expanded for each treatment record, and the cell line, compound identifier, bead batch, and detection plate are stored for each sample. The expression vectors for these treatment identifiers are then loaded from the Level 3 matrix.

For each treatment profile, control candidates are selected from the same cell line and the same experimental context. The primary rule matches cell line, bead batch, and detection plate. If no control exists for this exact match, a relaxed rule matches cell line and detection plate. If no suitable control exists, the treatment profile is removed from the candidate set. At this stage, the loader stores each treatment profile together with its candidate controls.

In the evaluation pipeline, data splitting is applied at the treatment profile level before final control matching. After the split, control pools for training and test are separated. Random matching is then applied within each split. In training, one treatment profile can be matched to several randomly sampled controls. In the reported experiments, the default number is 3 controls per training treatment. If the candidate pool contains fewer than 3 controls, sampling is done with replacement. In the test set, each treatment profile is matched to one control. The supervised target is the treatment response vector

$$y = x_{\text{trt}} - x_{\text{ctl}},$$

where x_{trt} is the treatment expression vector and x_{ctl} is the matched control expression vector. This construction removes much of the baseline cell state and focuses learning on the change caused by drug treatment. When controls are measured in a similar context, the subtraction also removes shared technical noise, such as batch or plate effects, and reduces unwanted variation in the target. Several steps are used to reduce the risk of data leakage. The split is created before the final treatment control pairs are sampled, so paired examples derived from the same treatment profile do not appear on both sides of the evaluation. Control pools are then separated for training and test, which prevents the same control profile from being reused in both sets. In the cold drug setting, test drugs are

removed from the training set. In the cold cell setting, test cell lines are removed from the training set. The test set also uses one matched control per treatment profile, which makes the evaluation stricter and easier to read.

Every training example is linked to a compound identifier and for each drug, we build a target vector over the selected genes. If a gene is an annotated target of the drug, its position in the vector is marked; otherwise it remains zero. Only treatment examples with usable drug target annotations are retained in settings that require target based features. Examples whose `pert_id` cannot be linked to an existing `target` field are removed.

After preprocessing, the dataset contains matched arrays for control gene expression, drug target vectors, and response targets. Identifiers for cell line, bead batch, detection plate, and profile ids are also retained. These identifiers are later used for evaluation and grouped analysis.

OmniPath graph preprocessing

The interaction graph is constructed from OmniPath resources that provide directed control or interaction links. The resulting graph is matched to the selected gene space. Each row describes a source gene and a target gene together with sign information when available.

OmniPath tables are provided in gene symbols. These symbols are mapped to the gene identifiers used by the expression matrices. Edges are kept only when both endpoints can be mapped and both genes are present in the selected gene set. This ensures that graph nodes, expression vectors, and drug target vectors all refer to the same ordered gene list.

The graph is represented as a directed edge list. When OmniPath provides sign annotations, each edge is assigned a sign value $w \in \{+1, -1\}$ for stimulation or inhibition. If sign information is missing, the edge can be excluded or treated as unknown, depending on the experiment setting.

An interaction may appear multiple times across sources. Duplicate edges are merged so that each ordered pair (u, v) appears once in the final graph. The resulting graph is stored as `edge_index`, a $2 \times E$ integer tensor, together with optional edge attributes such as signed weights. This representation is used directly by the graph neural network.

In the landmark gene setting used in this thesis, the resulting OmniPath graph contains 978 nodes and 1,627 directed edges after filtering and deduplication. The node set matches the 978 dimensional landmark expression space, so graph outputs align directly with the prediction targets. When the full gene space is used, the constructed graph contains 12,328 nodes and 26,166 directed edges under the same preprocessing settings.

Summary

After preprocessing, each modelling example contains a matched control expression profile x_{ctl} , a treatment response target $y = x_{\text{trt}} - x_{\text{ctl}}$, a drug feature vector aligned to the same gene order, and sample related identifiers such as cell line and profile ids. These sample level inputs are combined with a fixed OmniPath derived interaction graph.

Only treatment examples that can be linked to matched controls and usable drug features are retained for modelling in the relevant experiment setting. In practice, this means that the final dataset contains paired treatment control examples for which the response target and model inputs can both be constructed.

3

Theory

3.1 Deep neural networks

This section introduces only the neural network parts that are used later in the thesis. The baseline model relies on dense layers, nonlinear activations, dropout, normalization, and mini batch training. The graph model also uses residual blocks, layer normalization, and a position wise feed forward block.

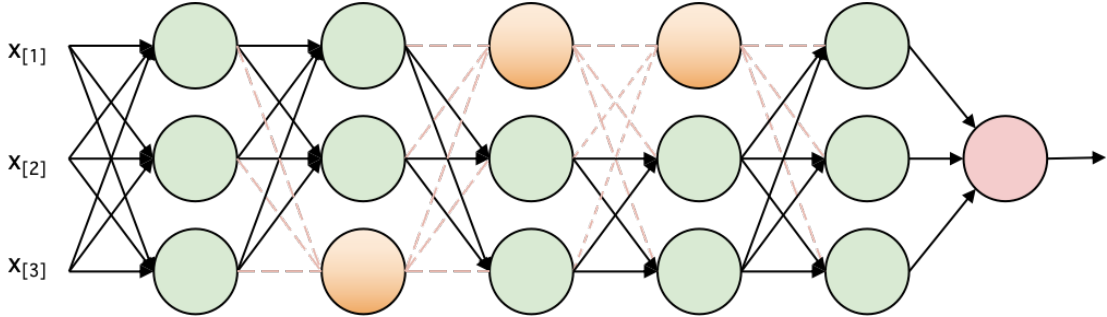


Figure 3.1: A Feed-Forward Network with several hidden layers. When an FFN contains several hidden layers, it can also be viewed as a deep neural network.

Dense layer

A dense layer maps an input vector to a new representation:

$$h = \sigma(Wx + b), \quad (3.1)$$

Here $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a weight matrix, $b \in \mathbb{R}^{d_{\text{out}}}$ is a bias vector, and $\sigma(\cdot)$ is an activation function. Stacking dense layers lets the model transform the input step by step.

Activation functions

The activation function adds nonlinearity. Without it, several dense layers would still behave like one linear map. The main choice in this thesis is ReLU,

$$\text{ReLU}(x) = \max(0, x), \quad (3.2)$$

which is used in the dense baseline. ReLU is simple and usually works well in hidden layers. For regression tasks, the final output layer often uses no activation function.

Stacked layers

If $h^{(0)} = x$, a network with L hidden layers can be written as

$$h^{(\ell+1)} = \sigma\left(W^{(\ell)}h^{(\ell)} + b^{(\ell)}\right), \quad \ell = 0, \dots, L-1. \quad (3.3)$$

The output of one layer becomes the input of the next, so deeper networks can build more expressive hidden representations.

Dropout

Dropout is a simple method for reducing overfitting [22]. During training, part of the hidden units are randomly turned off. If a hidden vector is $h \in \mathbb{R}^d$ and the dropout mask is $r \in \{0, 1\}^d$, the dropped representation can be written as

$$\tilde{h} = r \odot h, \quad (3.4)$$

where $\odot$ is element wise multiplication. If one entry of r is zero, the matching hidden unit is removed in that update.

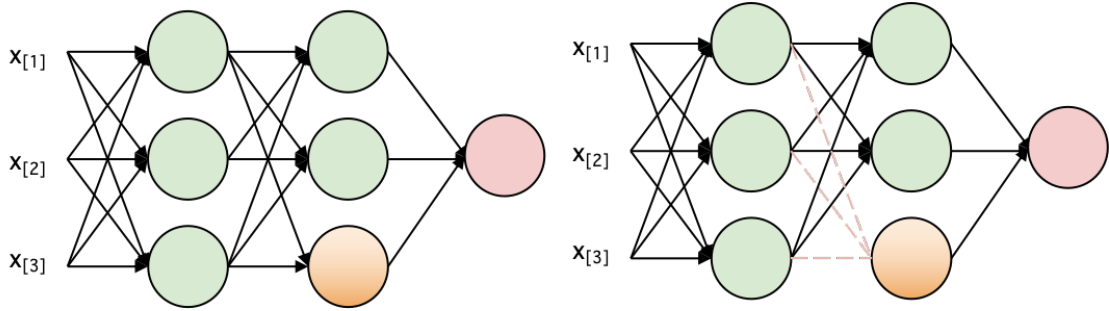


Figure 3.2: Dropout during training. Some hidden units are removed, so the model uses different paths in different updates.

This makes the network less dependent on a small set of hidden units and often helps it work better on new data. At test time, dropout is turned off.

Regularization

Regularization controls model complexity and helps reduce overfitting. A common form adds a penalty term to the data fitting loss:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \Omega(\theta), \quad (3.5)$$

Here θ is the set of model parameters, $\Omega(\theta)$ is a penalty term, and λ controls its strength. In practice, L2 regularization, also called weight decay, is the most common choice in deep learning.

Normalization layers

Normalization layers control the scale of hidden activations during training. Two common methods are batch normalization and layer normalization.

For a mini batch activation a , batch normalization subtracts a batch mean and divides by a batch standard deviation:

$$\text{BN}(a) = \gamma \frac{a - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}} + \beta, \quad (3.6)$$

Here γ and β are learned scale and shift parameters. Batch normalization uses statistics from the current mini batch. Layer normalization uses the same idea, but computes the mean and variance from one sample across its hidden features. For a hidden vector $h \in \mathbb{R}^d$, it can be written as

$$\text{LN}(h) = \gamma \frac{h - \mu_{\text{layer}}}{\sqrt{\sigma_{\text{layer}}^2 + \epsilon}} + \beta, \quad (3.7)$$

Here μ_{layer} and σ_{layer}^2 are computed from the features of the same sample. Batch normalization is used in the dense baseline, while layer normalization is used in the graph attention blocks.

Mini batches, learning rate, and optimizer

Training usually uses mini batches instead of the full dataset. The learning rate controls the step size of each update.

If the model parameters are written as θ and the loss on one mini batch is $\mathcal{L}_{\mathcal{B}}(\theta)$, a simple gradient based update is

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}(\theta), \quad (3.8)$$

where η is the learning rate.

Adam is a widely used optimizer [23]. It combines information from past gradients and squared gradients to produce smoother parameter updates, and it is used in the experiments reported later.

3.2 Graph neural networks

Graphs

A graph represents relations between a collection of objects. The objects are called nodes, and the relations are called edges. Mathematically, a graph is written as $G = (V, E)$, where V is the set of nodes and E is the set of edges. Each edge connects a pair of nodes. Figure 3.3 shows the main parts of a graph.

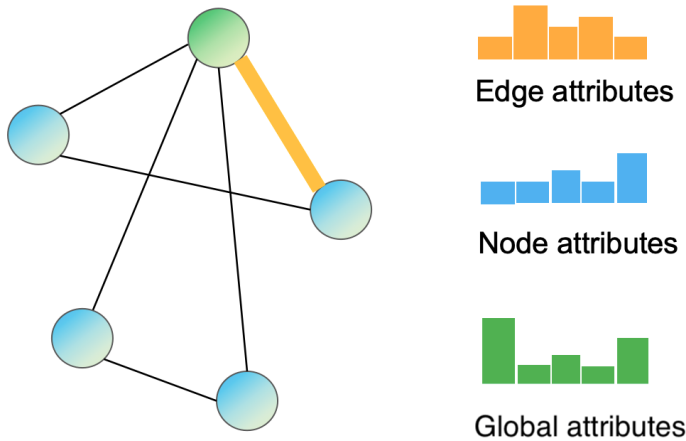


Figure 3.3: Main components of a graph. A graph can contain node attributes, edge attributes, and global information.

Inputs

The input to a graph neural network is not only the graph structure. It usually includes the graph connectivity together with feature information attached to nodes, edges, or the whole graph. Let a

graph have N nodes. Its connectivity can be written as an adjacency matrix $A \in \mathbb{R}^{N \times N}$. If node features have dimension F_0 , then the initial node feature matrix is

$$H^{(0)} \in \mathbb{R}^{N \times F_0}. \quad (3.9)$$

The pair $(A, H^{(0)})$ is the most basic input to a graph neural network.

In practice, the graph structure is often stored as an adjacency matrix. Figure 3.4 shows this idea. The graph on the left is written as a matrix on the right. If there is an edge from node N_j to node N_i , then the corresponding entry in the matrix is nonzero. If there is no edge, the entry is zero. This matrix form is convenient because the model can use it directly in message passing and neighborhood aggregation.

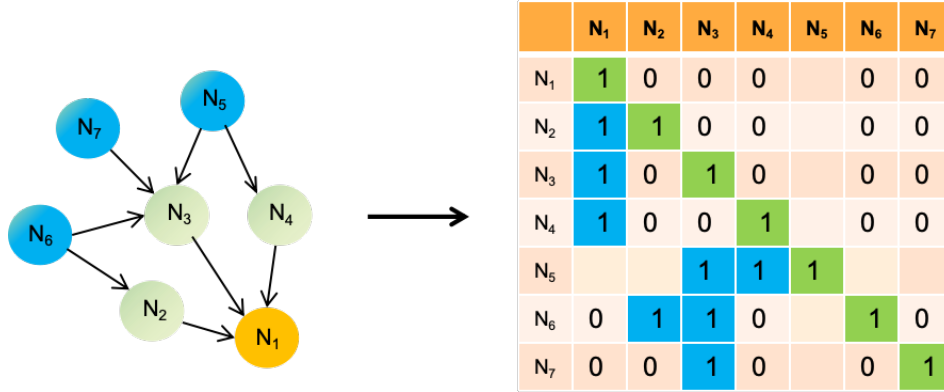


Figure 3.4: From a graph to its adjacency matrix. The graph structure is converted into a matrix form so that graph connections can be used in computation.

A node can have its own attributes, such as an expression value or a feature vector. An edge can also have attributes, such as direction, sign, or weight. Adding self loops is also common, because it lets the model keep part of the node's own state during aggregation. In biological graphs, such self loops should be understood as a model design choice rather than as a biological claim.

Graph direction

Graphs can be directed or undirected. In an undirected graph, the connection between two nodes has no direction. Information can move both ways. In a directed graph, each edge has a source node and a target node. So information flows from one node to another in a specific direction.

This difference matters in biology. Many molecular relations are directional. A regulator can affect a target gene, but the reverse relation may not hold. So a directed graph is often a more natural representation than an undirected graph. Once the graph structure is defined, a graph neural network still needs a rule for how information moves along these edges.

Message passing mechanism

This is the core idea that lets a graph neural network use the graph structure. The network works by moving data along the edges of the graph. In one layer, a node gets messages from its directly connected neighbors. For example, look at Figure 3.5. Node N_4 collects data from its direct neighbors N_1 , N_2 , and N_3 . This figure only shows what happens in one layer.

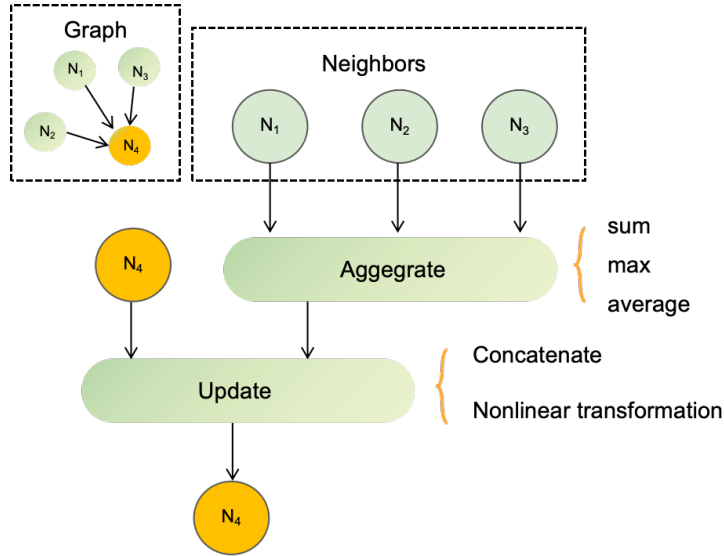


Figure 3.5: Message passing in a single layer. Messages from neighbors are grouped together and then used to update the target node.

After one layer, a node holds information from its direct neighbors. When more layers are stacked, the receptive field becomes larger and the node can also receive information from more distant parts of the graph.

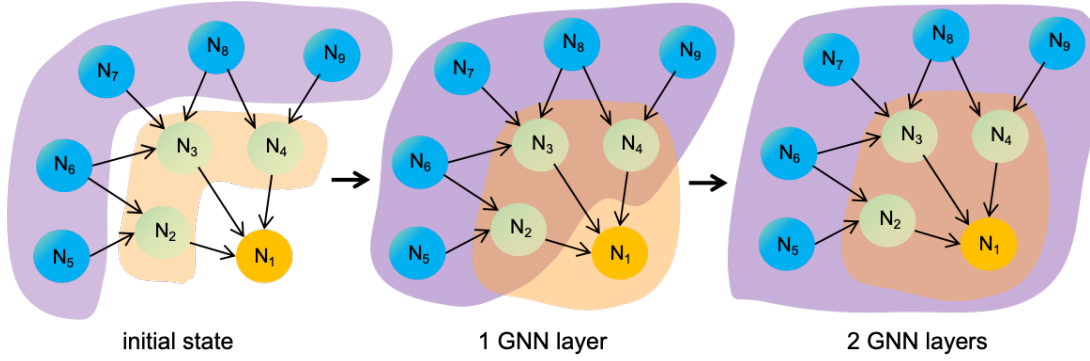


Figure 3.6: How message passing expands across layers. The yellow node N_1 is the target node. After one layer, it receives information from its one hop neighbors N_2 , N_3 , and N_4 . After two layers, information from the two hop neighbors N_5 to N_9 can also reach N_1 through the intermediate nodes.

Figure 3.6 shows this idea. In the first layer, node N_1 only uses the states of its one hop neighbors. In the second layer, those neighbors already contain information from nodes that are two hops away. This is why stacking local message passing layers still expands the effective range of information flow.

Message passing can be separated into two steps. Aggregation groups the incoming neighbor signals, and update uses the grouped message to compute a new node state.

Aggregation

The first step is aggregation. This step combines data from neighbor nodes into one single vector. We write a general aggregation operator $\square$ at layer ℓ like this:

$$m_{\mathcal{N}(v)}^{(\ell)} = \square_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)} \quad (3.10)$$

Here, the term $m_{u \rightarrow v}^{(\ell)}$ is a message from neighbor u to node v . The grouped message vector $m_{\mathcal{N}(v)}^{(\ell)}$ collects the total signal from the local neighbors.

Because a graph does not have a fixed neighbor order, the aggregation rule must not depend on neighbor order. Common choices are sum, mean, and max.

Update

After aggregation, the network updates the target node. This step uses the grouped neighbor data to make a new node state. An update at layer ℓ uses a learned function $\psi^{(\ell)}$. We write it like this:

$$h_v^{(\ell+1)} = \psi^{(\ell)}\left(h_v^{(\ell)}, m_{\mathcal{N}(v)}^{(\ell)}\right), \quad (3.11)$$

Here, $h_v^{(\ell)}$ is the state of node v at layer ℓ . If the next layer has size $F_{\ell+1}$, then the new state is $h_v^{(\ell+1)} \in \mathbb{R}^{F_{\ell+1}}$. The update function takes the target node's old state and the grouped message $m_{\mathcal{N}(v)}^{(\ell)}$. Then it combines them to make the new state $h_v^{(\ell+1)}$.

The update function is the step that actually changes the node state. For a mini batch of size B , the layerwise node tensor has shape

$$H^{(\ell)} \in \mathbb{R}^{B \times N \times F_{\ell}}. \quad (3.12)$$

In this batch, the graph structure matrix $A \in \mathbb{R}^{N \times N}$ is the same for all samples. The model repeats these aggregation and update steps for several layers. After that, it gets the final node states.

The final node representation matrix is $H^{(L)} \in \mathbb{R}^{N \times F_L}$. What happens next depends on the output form of the model.

Output

Graph neural networks can produce outputs at different levels. A node level output gives one prediction for each node. This is used in node classification and node regression. An edge level output gives one prediction for a pair of nodes or for an existing edge. This is common in link prediction and edge classification. A graph level output gives one prediction for the whole graph. This is often used in graph classification or graph regression.

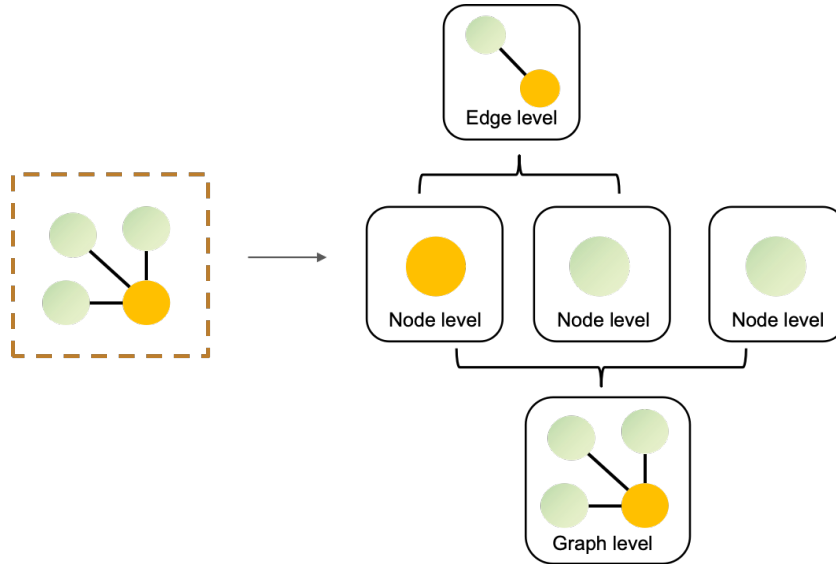


Figure 3.7: Three common GNN output levels. A GNN can produce one output for each node, one output for a selected edge, or one output for the whole graph after a readout step.

The output level determines how the final hidden states are used. For node level output, the model applies a predictor to each row of $H^{(L)}$. So each node keeps its own final representation and receives

its own output. For edge level output, the model combines the representations of two nodes, and sometimes also an edge feature, to predict a relation between them. For graph level output, the model must reduce all node representations to one graph representation before the final prediction. This reduction step is usually called pooling or readout. Common choices are sum pooling, mean pooling, and max pooling. Sum pooling adds node vectors and can reflect graph size. Mean pooling gives a more normalized summary. Max pooling keeps the strongest value in each feature dimension. Some models also use attention based pooling, where the model learns which nodes should contribute more to the graph representation.

The output level is related to the final task, but the task itself can still vary within the same level. For example, node level output can be used for either classification or regression. Graph level output can also be used for either classification or regression. In this thesis, the model predicts a continuous drug response value for each gene. Each gene is treated as a node in the biological graph. So the thesis uses a node level output, and the task is node level regression.

Graph Convolutional Networks (GCN)

Figure 3.8 shows a simple view of graph convolutional updates. Graph Convolutional Networks, or GCNs, are a standard GNN architecture [11]. The main idea is simple. Each node updates its representation by combining its own information with information from nearby nodes. The graph structure gives the weights. So connected nodes can share information.

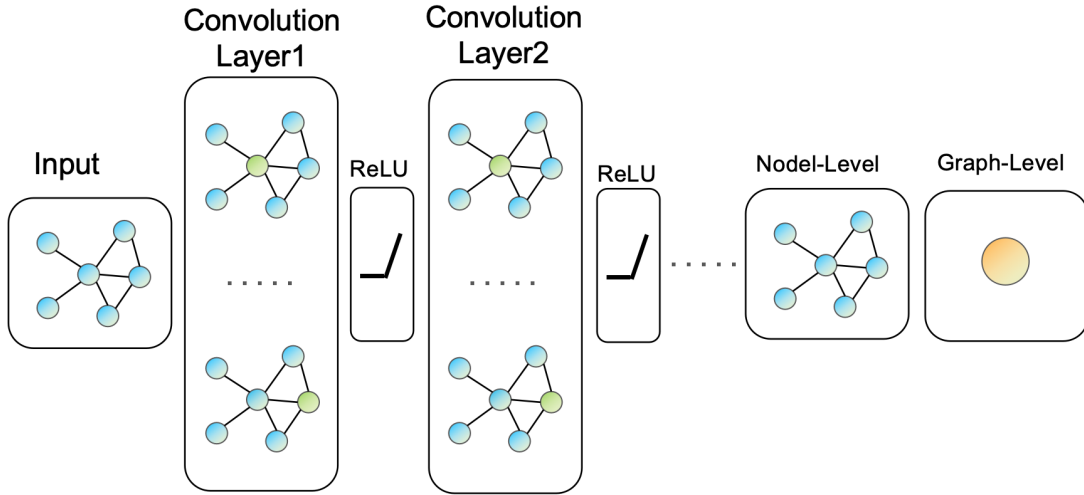


Figure 3.8: A simple view of Graph Convolutional Networks. The input graph is processed by graph convolution layers, and the learned node representations are used for prediction.

A GCN usually has an input layer, one or more graph convolution layers, and an output layer. The input layer provides the initial node feature vectors. Each graph convolution layer updates the node representations by mixing information from neighbouring nodes. The output layer uses the final node representations for prediction. So deeper GCNs can use information from a larger part of the graph.

The GCN used in this thesis follows the common formulation of Kipf and Welling [11]. This formulation gives a simple and efficient graph convolution rule. Instead of using a more complicated graph transformation, it uses a normalized update matrix that is easy to compute in practice.

Let the graph have N nodes. Then the adjacency matrix is $A \in \mathbb{R}^{N \times N}$. Let $I \in \mathbb{R}^{N \times N}$ be the identity matrix. Then $\tilde{A} = A + I \in \mathbb{R}^{N \times N}$ adds self loops to the graph. Let $\tilde{D} \in \mathbb{R}^{N \times N}$ be the degree matrix of $\tilde{A}$. Then the normalized update matrix is

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}. \quad (3.13)$$

A GCN layer computes

$$H^{(\ell+1)} = \sigma(\hat{A}H^{(\ell)}W^{(\ell)}), \quad (3.14)$$

where $\hat{A} \in \mathbb{R}^{N \times N}$, $H^{(\ell)} \in \mathbb{R}^{N \times F_\ell}$, $W^{(\ell)} \in \mathbb{R}^{F_\ell \times F_{\ell+1}}$, and $H^{(\ell+1)} \in \mathbb{R}^{N \times F_{\ell+1}}$. Here F_ℓ is the feature dimension of the node representation before the layer, and $F_{\ell+1}$ is the feature dimension after the layer. In other words, each row of $H^{(\ell)}$ is the feature vector of one node at layer ℓ .

This equation first transforms the node representations and then mixes information across neighbouring nodes. Because of the self loops in $\hat{A}$, a node also keeps part of its own information in the update. GCNs are simple and widely used, but they treat neighbours with a fixed normalized weighting rule. This can be limiting when some neighbours are much more important than others, and very deep GCNs can also suffer from over smoothing.

Graph Attention Networks (GAT)

Graph Attention Networks (GATs) give different weights to different neighbours [8]. Figure 3.9 shows how the attention weights are computed.

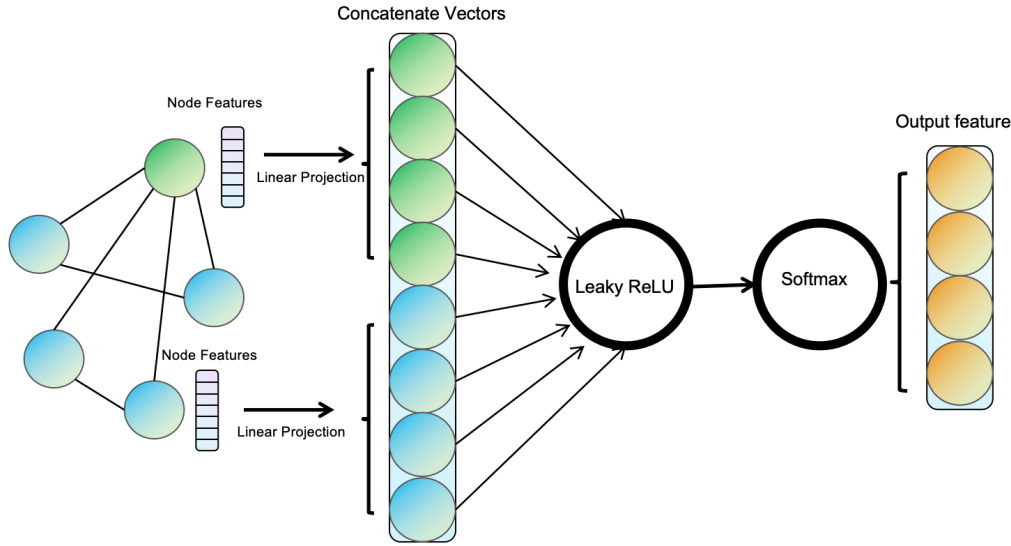


Figure 3.9: Computation of attention scores and normalized attention weights in a graph attention layer. Node features are projected, compared, passed through LeakyReLU, and then normalized by softmax.

GAT differs from GCN because it does not use one fixed normalized weighting rule for all neighbours. Instead, it learns how important each neighbour is for the current target node. This makes the aggregation more flexible.

GAT layer

Let $h_u^{(\ell)}, h_v^{(\ell)} \in \mathbb{R}^{F_\ell}$ be node representations at layer ℓ . A GAT layer first applies a learned projection

$$z_u^{(\ell)} = W^{(\ell)} h_u^{(\ell)}, \quad (3.15)$$

Here $W^{(\ell)} \in \mathbb{R}^{F_{\ell+1} \times F_\ell}$. It then computes an attention score

$$e_{u \rightarrow v} = \text{LeakyReLU}\left(a^\top \begin{bmatrix} z_u^{(\ell)} \| z_v^{(\ell)} \end{bmatrix}\right), \quad (3.16)$$

Here $a \in \mathbb{R}^{2F_{\ell+1}}$. The LeakyReLU function is

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \quad (3.17)$$

Here α is a small positive constant. The normalized attention weight is

$$\alpha_{u \rightarrow v} = \text{softmax}_u(e_{u \rightarrow v}). \quad (3.18)$$

The new node representation is

$$h_v^{(\ell+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{u \rightarrow v} z_u^{(\ell)} \right). \quad (3.19)$$

These attention weights are learned from data. They depend on the features of both the source node and the target node, so the same neighbour can receive different importance in different local settings.

Multi-head attention

GAT can also use several attention heads in parallel. If the model uses K heads, each head has its own projection and attention parameters. The outputs can be concatenated or averaged, depending on the layer design.

If each head outputs F_{head} features and the model concatenates the heads, then the output width is

$$F_{\ell+1} = K F_{\text{head}}. \quad (3.20)$$

This relation is useful when model dimensions are chosen in practice.

Multi head attention gives the model several views of the same neighborhood and can make the representation richer than using one attention rule alone.

Residual connections

Residual connections add the input of a block back to its output [24]. If a block computes a transformation $f(x)$, the residual form returns

$$z = x + f(x). \quad (3.21)$$

Figure 3.10 shows this idea. The input x goes through the stacked layers and also follows a shortcut path. These two paths are added at the end.

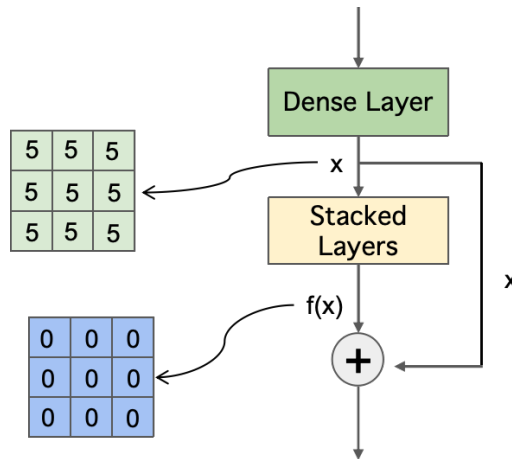


Figure 3.10: A residual connection. The input follows both the main stacked layers and a shortcut path, and the two outputs are added together.

This design keeps a direct path from the input to the output of the block. It helps preserve useful information from earlier layers and often makes optimization more stable in deeper models.

Position wise feed forward network (FFN)

In Transformer related models, an attention layer is often followed by an FFN [25]. Here FFN means a Position wise Feed forward Network. This use of FFN is different from the FFN at the beginning of this chapter where FFN referred to a full feed forward network. Here it refers to a small block applied to each node representation after attention.

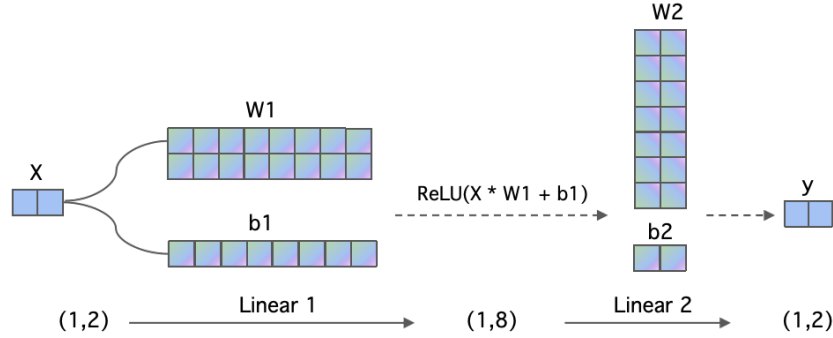


Figure 3.11: A Position-wise Feed-Forward Network block. It consists of two linear transformations with a nonlinear activation function in between.

Figure 3.11 shows the structure of this block. It applies two linear layers to the input node state with an activation function in between. If the node state after attention is h_v , the model computes the FFN output like this:

$$\text{FFN}(h_v) = W_2 \sigma(W_1 h_v + b_1) + b_2. \quad (3.22)$$

Here W_1 and W_2 are the weight matrices and b_1, b_2 are the bias vectors. The first linear layer expands the feature dimension from d_{model} to a larger dimension d_{ff} , and the second projects it back to d_{model} . The same parameters are shared across all nodes, **but the transformation is applied to each node independently.**

GAT attention block

In the original GAT definition, the core operation is the attention based neighborhood aggregation described above [8]. In many later attention based models, the attention layer is followed by a position wise FFN, with residual connections and layer normalization around these sublayers [25, 24, 26]. This design became widely known through the Transformer architecture [25] and was later adopted in graph Transformer style models such as Graphormer and GraphGPS [13, 14].

Each part in this block has a different role. Attention mixes information across neighboring nodes, the FFN further transforms each node representation, residual connections preserve useful earlier information, and layer normalization helps keep training stable [25, 24, 26]. ~~This design was later adopted in graph Transformer style models such as Graphormer and GraphGPS [13, 14].~~

Advantage

GAT can be more useful when the graph is noisy or incomplete. In many real datasets, some edges are weak, indirect, or uncertain. A fixed aggregation rule treats all neighbours in a similar way after normalization. Attention can reduce the influence of less informative neighbours and give more weight to neighbours that appear more relevant for the current prediction. This does not remove graph noise, but it can make the model more robust to it.

Uncertainty estimation

Predictions should not always be interpreted as equally reliable. Uncertainty estimation is used to represent this difference. Instead of giving only one predicted value, the model can also describe how certain or uncertain that prediction is. In this thesis, the uncertainty mainly refers to input dependent noise in the prediction target rather than uncertainty in the model parameters.

Gaussian assumption

A regression target does not need to be treated as one fixed value only. It can also be modeled as a random variable with a probability distribution. A Gaussian assumption is common in regression because the target is continuous and the model needs to describe both a central value and the spread around that value. A common choice for continuous prediction is a Gaussian distribution:

$$y \sim \mathcal{N}(\mu, \sigma^2). \quad (3.23)$$

Here μ is the expected value of the target and σ^2 describes the spread around that value. A small variance means that the target is expected to stay close to the mean, while a large variance means that the prediction is treated as less certain.

Predicting mean and variance

Under this view, the model predicts two values instead of one. The first is the mean prediction μ , and the second is a variance related term. In practice, models often predict $\log \sigma^2$ instead of σ^2 directly, because this is easier to train, more stable, and still defines a positive variance after exponential operation [27].

Negative log likelihood

If the target is modeled with a Gaussian distribution, a natural training objective is the negative log likelihood (NLL). its general form is

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \left[\log \sigma^2 + \frac{(y - \mu)^2}{\sigma^2} \right]. \quad (3.24)$$

The squared error term encourages the mean prediction to stay close to the target. The $\log \sigma^2$ term penalizes unnecessarily large uncertainty. Together, these two terms prevent the model from assigning high variance everywhere while still allowing uncertain predictions when needed.

Summary

Uncertainty estimation is useful when some targets are easier to predict than others. In biological data, some responses are clearer and more consistent, while others are noisier or more context dependent. Predicting both a mean and an uncertainty lets the model represent different confidence levels across genes or samples instead of treating all predictions as equally certain.

4

Methodology

4.1 Target

This thesis studies prediction of gene expression changes after drug treatment. Let G be the number of genes in the selected gene set. For each paired sample, the input includes a matched control profile $x_{\text{ctl}} \in \mathbb{R}^G$ and a treated profile $x_{\text{trt}} \in \mathbb{R}^G$. The training target is their difference:

$$y = x_{\text{trt}} - x_{\text{ctl}} \in \mathbb{R}^G. \quad (4.1)$$

The model predicts a response vector $\hat{y} \in \mathbb{R}^G$.

When processed in a mini-batch of size B , the target tensor has the shape $Y \in \mathbb{R}^{B \times G}$ and the corresponding model prediction has the shape $\hat{Y} \in \mathbb{R}^{B \times G}$.

4.2 Inputs

A matched control profile, a drug representation, and cell related information are used as the three main inputs. These inputs use the same gene order, so the control vector, the drug vector, the graph matrix, and the predicted response all refer to the same gene positions.

Control data

The matched control profile $x_{\text{ctl}} \in \mathbb{R}^G$ shows the baseline state of the cell before drug treatment. It is important because the same drug can produce different responses when the starting state is different. Some genes may already be active, while others may be weak. By using the control profile directly, the model can base its prediction on the measured state of the sample instead of relying on the drug input alone.

In the main setup, $G = 978$ landmark genes are used, so each control sample is a 978 dimensional vector of continuous values. For a mini batch of size B , the control tensor has shape $X_{\text{ctl}} \in \mathbb{R}^{B \times G}$. As shown in Figure 4.1, each gene value is mapped to the corresponding graph node and projected to the hidden dimension. This gives every node a starting feature based on the measured control expression.

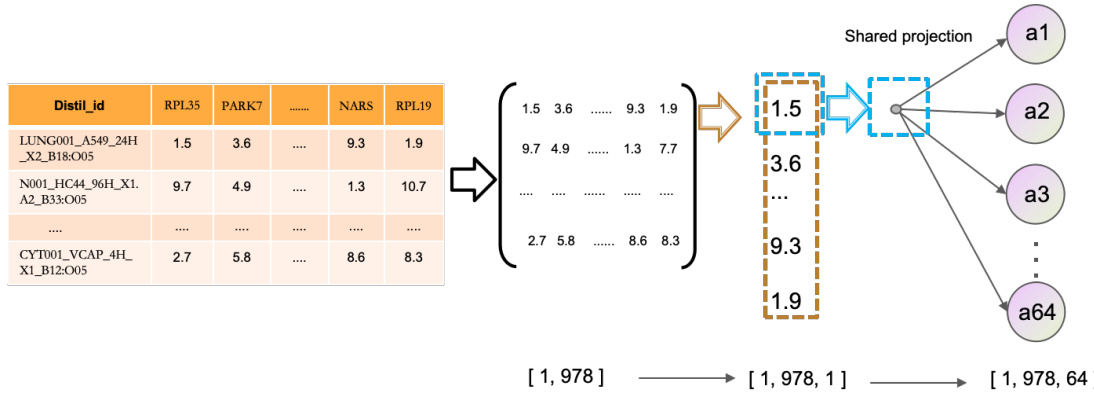


Figure 4.1: Control input and its node wise projection. Each sample is represented as a 978 dimensional gene expression vector aligned to the gene order. The same shared projection layer maps each gene wise scalar input to a 64 dimensional node feature.

Drug target

The main drug input is a binary target vector $x_{\text{drug}} \in \mathbb{R}^G$. Each entry is one when the corresponding gene is a known target of the drug, and zero otherwise. Some studies use richer drug features, but the main experiments in this thesis use only this target based vector. For a mini batch, the drug input has shape $X_{\text{drug}} \in \mathbb{R}^{B \times G}$.

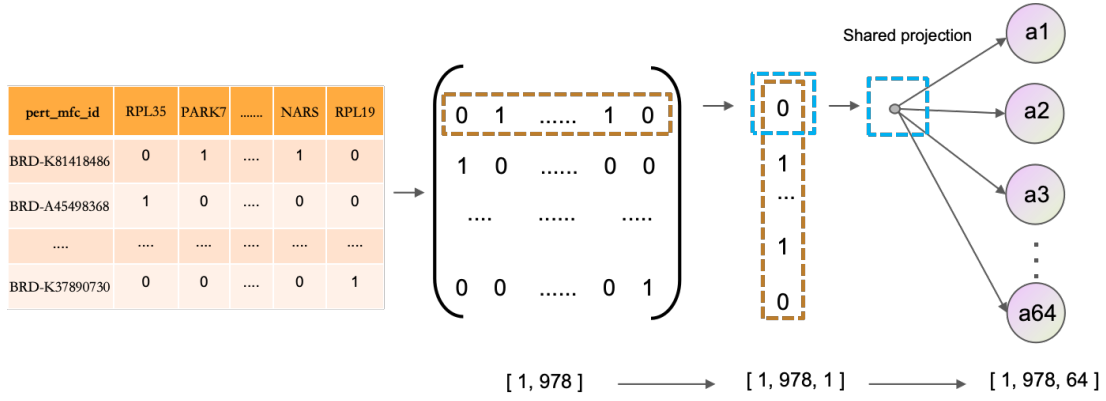


Figure 4.2: Drug target input and its node wise projection. Each sample is represented as a binary target vector aligned to the gene order. After adding one extra feature dimension, the same shared projection layer maps each gene wise scalar input to a 64 dimensional node feature.

Biological graph

The proposed architecture uses a directed biological graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ derived from the OmniPath database. Each node represents one gene, and each directed edge represents a known effect from one gene to another. If gene u can affect gene v , the graph includes a directed edge from u to v . Since the selected gene set contains G genes, the graph can be written as a matrix $A \in \mathbb{R}^{G \times G}$.

The graph stores not only whether two genes are connected, but also whether the effect is activating or inhibitory. Here gene u is the source node and gene v is the target node. If gene u activates gene v , the edge from u to v is assigned a positive sign. If gene u inhibits gene v , the edge is assigned a negative sign. In this way, the graph tells the model not only where information can flow, but also whether the incoming signal should increase or oppose the current state of the target gene.

This sign can be written as

$$s_{vu} = \begin{cases} 1, & \text{if gene } u \text{ activates gene } v, \\ -1, & \text{if gene } u \text{ inhibits gene } v. \end{cases} \quad (4.2)$$

The message from gene u to gene v is then adjusted by this sign:

$$m_{vu} = s_{vu} h_u. \quad (4.3)$$

Here h_u is the current representation of gene u . A positive sign keeps the message in the same direction, while a negative sign reverses it.

Cell information

The first approach uses a cell identity embedding. This is a very direct way to tell the model which cell line the sample comes from. Each cell line is assigned one learned vector in $\mathbb{R}^F$, so all samples from the same cell line share the same representation. This can work well because cell background has a strong effect on gene expression. In other words, knowing the cell line already gives the model important information about the likely response pattern. At the same time, this also means that the model can rely heavily on the cell identity itself. The limitation is that samples from the same cell line are treated as if they were in the same state, even when their measured baseline profiles are different.

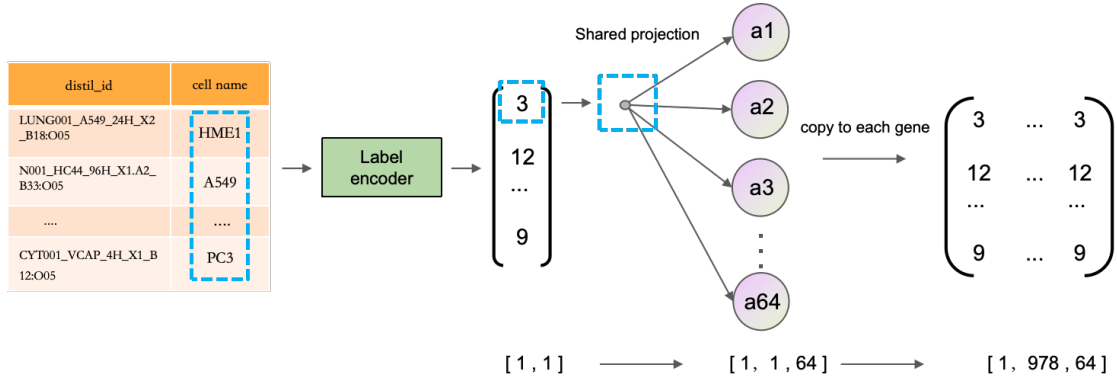


Figure 4.3: Cell identity input and its embedding. Each cell line name is first mapped to an integer index by a label encoder, and then a learnable embedding layer maps the index to a 64 dimensional cell representation. The same cell embedding is shared by all samples from the same cell line.

Another approach uses cell context. Cell context means the state of one sample before the drug is applied. This information can be taken from the matched control profile [17, 28], because the control profile is measured before treatment and shows which genes are already high or low in that sample. In this sense, it gives a snapshot of the sample before treatment. Even samples from the same cell line can have different baseline states, and these differences appear in the control expression values. Instead of using a fixed cell label, the model reads the full control profile x_{ctl} and converts it into a sample specific context vector $c_{\text{ctx}} \in \mathbb{R}^F$.

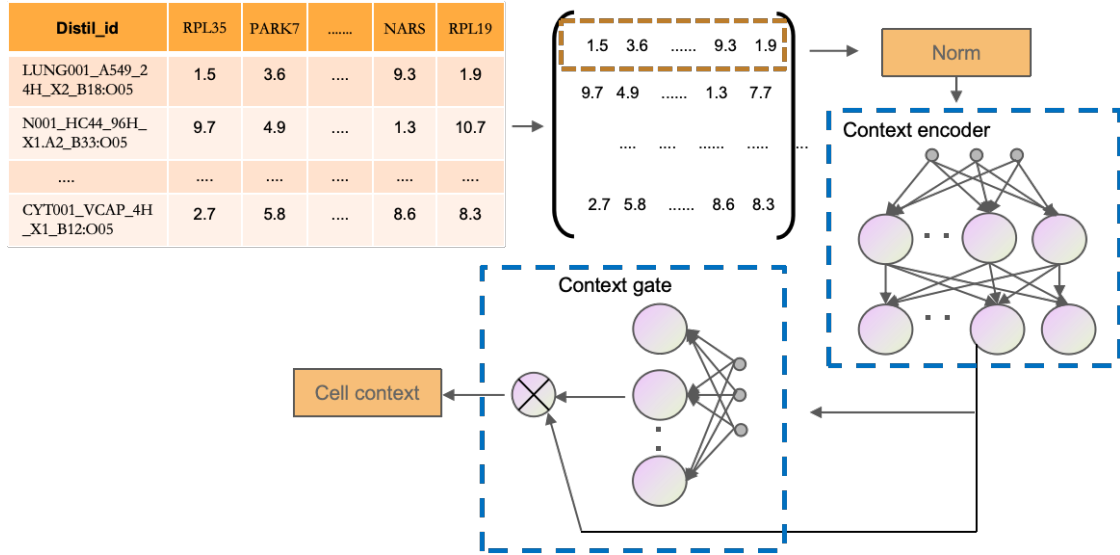


Figure 4.4: Control derived cell context. The control profile is normalized and encoded into a sample specific context vector. This context summarizes the baseline expression state of the sample before drug perturbation.

In CAGNN, the cell context vector c_{ctx} is copied to all G gene nodes and added to their initial node features. This lets the graph network start from node features that reflect the current sample state before perturbation [28].

Summary

Many studies in perturbation prediction use extra input features, such as molecular fingerprints, gene ontology features, or mutation profiles [18, 28, 29, 30]. The main experiments in this thesis use a simpler input setting. All models use the matched control profile and the drug representation. The proposed model also uses cell context information, while the two baselines do not.

This simpler design makes the comparison easier to follow. With fewer extra inputs, the comparison focuses more directly on the matched control profile, the drug input, and the added cell context.

4.3 Base model

Overview

Figure 4.5 shows the baseline model. It is a fully connected design [18], without using a biological interaction graph. The output layer predicts a mean response and an uncertainty related value for each gene.

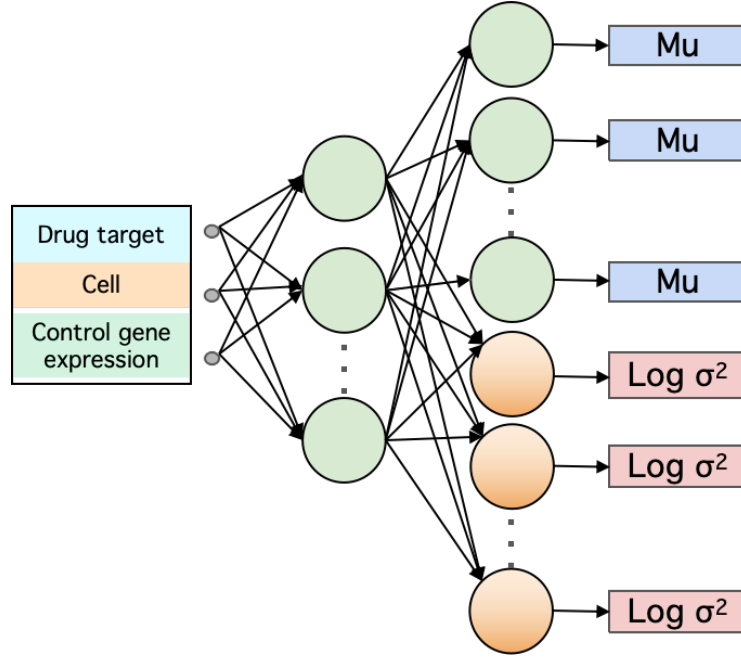


Figure 4.5: Structure of the baseline model. Control gene expression and drug features are concatenated and passed through a feed forward network to predict a mean response and a log variance for each gene.

The baseline model uses the control profile and drug input introduced above. The two vectors are concatenated:

$$z = [x_{\text{ctl}} \| x_{\text{drug}}] \in \mathbb{R}^{2G}. \quad (4.4)$$

For a mini batch, the same input is written as $Z \in \mathbb{R}^{B \times 2G}$. The model then applies two fully connected blocks. Each block uses a dense layer, batch normalization, ReLU, and dropout. The output stage predicts a mean value and a log variance value for each gene. For one sample, the final output can be written as

$$[\mu, \log \sigma^2] \in \mathbb{R}^{G \times 2}. \quad (4.5)$$

At batch level, the final output has shape $\hat{Y} \in \mathbb{R}^{B \times G \times 2}$.

Loss function

The baseline model predicts both a mean response and a log variance. The prediction target is the same response vector used in the thesis,

$$y = x_{\text{trt}} - x_{\text{ctl}}, \quad (4.6)$$

and the base error term is a Gaussian negative log likelihood. The likelihood term is

$$\mathcal{L}_{\text{nll}}^{\text{base}} = \frac{1}{BG} \sum_{i=1}^B \sum_{g=1}^G \frac{1}{2} \left[\log \sigma_{ig}^2 + \frac{(y_{ig} - \mu_{ig})^2}{\sigma_{ig}^2} \right]. \quad (4.7)$$

where B is the mini batch size. Here μ_{ig} is the predicted mean response and σ_{ig}^2 is the predicted variance for gene g in sample i .

The baseline also includes the same sample wise Pearson correlation term used in CAGNN,

$$\mathcal{L}_{\text{pcc}}^{\text{base}} = 1 - \frac{1}{B} \sum_{i=1}^B \text{corr}(y_i, \mu_i), \quad (4.8)$$

so the final objective is

$$\mathcal{L}_{\text{base}} = \mathcal{L}_{\text{nll}}^{\text{base}} + \lambda_{\text{pcc}} \mathcal{L}_{\text{pcc}}^{\text{base}}. \quad (4.9)$$

In the reported baseline setting, $\lambda_{\text{pcc}} = 5$. This keeps the baseline loss aligned with the proposed model, because both models are trained to reduce point wise error while also preserving the overall response pattern across genes.

Training settings

The baseline model is trained on the same paired perturbation data and split settings used for the proposed model. Optimization uses mini batches and Adam. Table 4.1 summarizes the baseline configuration used in the experiments.

Table 4.1: Main configuration of the baseline model.

Setting	Value
Optimizer	Adam
Learning rate	5×10^{-4}
Batch size	64
Training epochs	50
Hidden dimension	1024
Number of dense blocks	2
Dropout	0.2
PCC weight λ_{pcc}	5
Output	μ and $\log \sigma^2$

4.4 GSNN baseline

GSNN [17] is the graph based baseline in this thesis. It is evaluated on the same paired perturbation data and the same biological graph as the proposed model, but the computation is different.

Overview

GSNN uses only the matched control profile and the drug input vector,

$$x_{\text{GSNN}} = [x_{\text{ctl}}; x_{\text{drug}}] \in \mathbb{R}^{2G}. \quad (4.10)$$

This baseline does not use the extra cell context input used by the proposed model. The training target is the response vector

$$y = x_{\text{trt}} - x_{\text{ctl}}, \quad (4.11)$$

so the model predicts a gene wise response vector rather than the raw treated expression profile.

The implementation builds a graph with several node groups. For each gene, there is one control input node, one drug input node, one internal function node, and one output node. The control and drug inputs connect to the corresponding function node. The function nodes are connected by the biological graph. Each function node then connects to one output node, which produces the predicted response for that gene.

The internal update rule is not the same as the graph attention mechanism used in the proposed model. In this model, GSNN uses sparse linear updates inside residual blocks, and the graph decides which connections are allowed. Information from incoming edges is collected into hidden channels of each function node, processed by a nonlinearity and a node wise MLP, and then sent back to the outgoing edges. GSNN uses the graph during message passing, but it does not learn a separate importance weight for every edge.

The output of GSNN is a gene level prediction vector with the same dimension as y . Compared with the proposed model, it does not include control derived cell context or graph attention based message passing.

Loss function

GSNN is trained with a response prediction loss only. If the prediction for sample i is $\hat{y}_i \in \mathbb{R}^G$ and the target is $y_i \in \mathbb{R}^G$, the loss is

$$\mathcal{L}_{\text{GSNN}} = \frac{1}{BG} \sum_{i=1}^B \sum_{g=1}^G (y_{ig} - \hat{y}_{ig})^2. \quad (4.12)$$

This objective only trains the response prediction itself. It does not include a separate uncertainty term, a PCC term, or a counterfactual hinge term.

Training settings

GSNN is trained on the same paired perturbation data and under the same split settings as the proposed model. The same response target $y = x_{\text{trt}} - x_{\text{ctl}}$ is used. Optimization uses mini batches and standard gradient based training. This keeps the comparison focused on the model design rather than on differences in the data pipeline.

4.5 Proposed model

Overview

Figure 4.6 shows the proposed model. It uses a graph attention network on a biological interaction graph and predicts one response value for each gene.

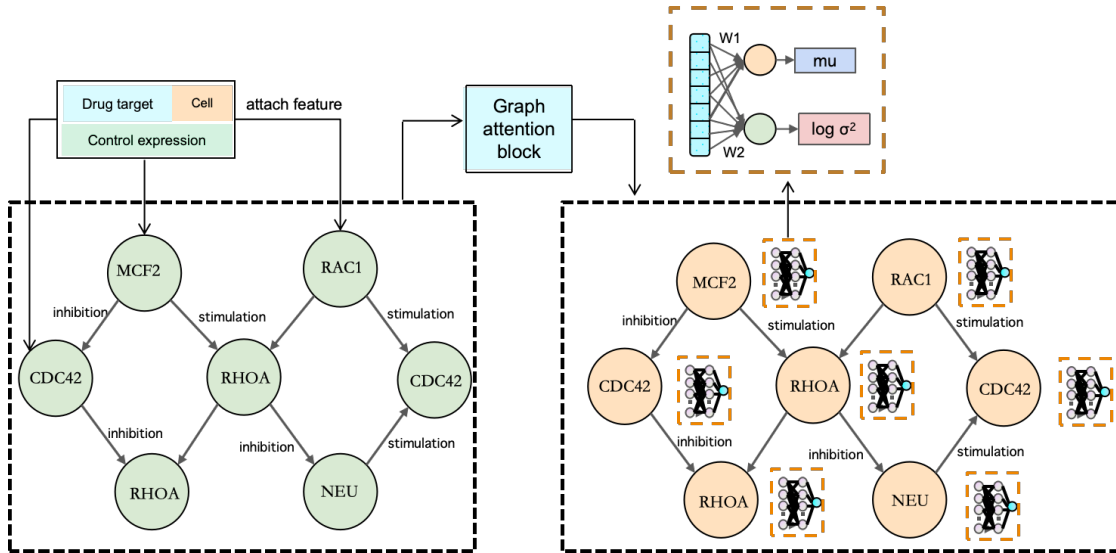


Figure 4.6: Structure of the proposed model. The model combines the biological interaction graph, the control profile, and drug target information to predict the gene wise response.

The graph, node features, and cell information are introduced in the input section above. The model combines these inputs to predict the response profile. The same uncertainty aware graph backbone is used throughout the CAGNN analyses in this thesis. The ablation analysis changes how cell information is provided, but the model still predicts both a mean response and a gene wise uncertainty value.

Node features

The proposed model starts with one node per gene. The control profile provides one scalar input for each node, and the drug target vector provides another scalar input. These inputs are projected to a

hidden dimension F . In the reported model, $F = 64$. The initial node representation matrix is

$$H^{(0)} \in \mathbb{R}^{G \times F}. \quad (4.13)$$

The control input and the drug target input are embedded separately, and the cell context is added as an extra term:

$$H^{(0)} = \phi_{\text{ctl}}(x_{\text{ctl}}) + \phi_{\text{drug}}(x_{\text{drug}}) + c_{\text{ctx}}. \quad (4.14)$$

For a mini batch, the node feature tensor becomes

$$H^{(0)} \in \mathbb{R}^{B \times G \times F}. \quad (4.15)$$

When cell information is added here, each gene node starts with control and drug signals together with information about the current cell state. This means that message passing begins from a representation that already reflects the sample context.

Graph multi-head attention block

The backbone contains $L = 4$ graph attention layers. Each layer uses multi head attention with $K = 4$ heads. Since the hidden width is $F = 64$, each head outputs $64/4 = 16$ features and the concatenated output returns to dimension 64. Cell information is not inserted directly into the attention score. Instead, the cell context vector is added to the node features before message passing starts.

The general GAT computation and the role of residual connections, layer normalization, and position wise FFNs are introduced in the theory chapter. In the proposed model, each block uses standard multi head graph attention for neighbor aggregation, followed by dropout, residual addition, and layer normalization. A position wise FFN is then applied, followed again by dropout, residual addition, and layer normalization. The FFN follows the form introduced in (3.22).

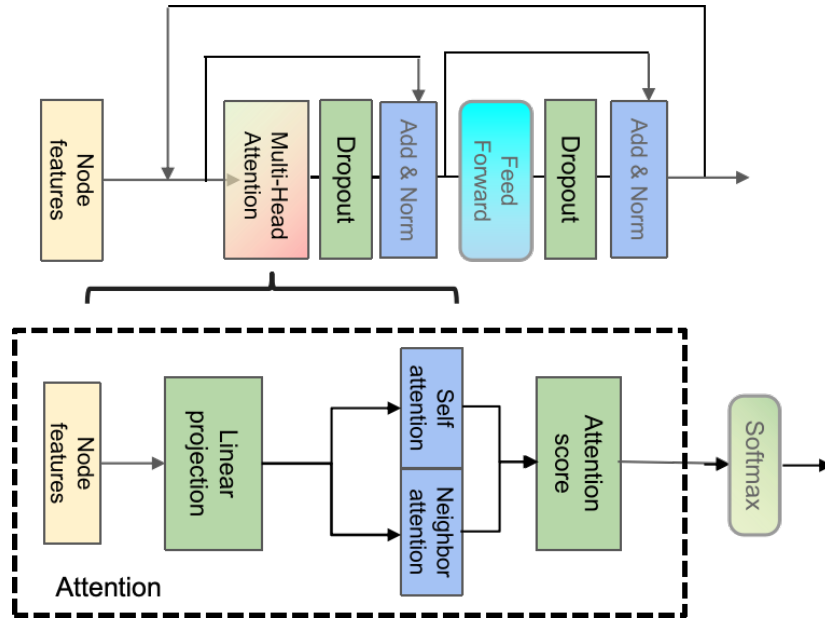


Figure 4.7: Flow of one graph attention block used in the proposed model. The input node features are linearly projected, standard graph attention scores are computed and normalized with softmax, and the multi head output is followed by dropout, residual addition, layer normalization, and a position wise feed forward network.

Figure 4.7 shows the computation in one graph block. The block uses the node representations prepared in the input stage, which already contain the sample context. It then applies standard graph attention for neighbor aggregation. When several graph attention layers are stacked, the output of one block becomes the input to the next block.

Output layer

After the last graph layer, each node has a final hidden vector $h_v^{(L)} \in \mathbb{R}^F$. The output layer uses two parallel linear readouts. One predicts the mean response,

$$\mu_v = w_\mu^\top h_v^{(L)} + b_\mu, \quad (4.16)$$

and the other predicts a log variance term,

$$\log \sigma_v^2 = w_\sigma^\top h_v^{(L)} + b_\sigma, \quad (4.17)$$

where $w_\mu, w_\sigma \in \mathbb{R}^F$ and $b_\mu, b_\sigma \in \mathbb{R}$. Stacking these outputs over all genes gives the full mean prediction $\mu \in \mathbb{R}^G$ and the full log variance prediction $\log \sigma^2 \in \mathbb{R}^G$.

Both outputs are computed from the same final hidden vector $h_v^{(L)}$, but they do not use the same parameters. The mean prediction uses w_μ, b_μ , while the log variance prediction uses w_σ, b_σ . This means that the same node representation is projected in two different ways, so the model can learn one mapping for the expected response and another for the uncertainty related value.

Loss function

The proposed model is trained with a combined objective. Part of this design is the same as in the baseline. Both models predict a mean response and a log variance and use a Gaussian negative log likelihood together with a sample wise PCC term. The probabilistic interpretation of the output and the general Gaussian negative log likelihood are introduced in the theory chapter in (3.23) and (3.24). For sample i and gene g , the target is compared with the predicted mean μ_{ig} and predicted log variance $\log \sigma_{ig}^2$, so the likelihood term is

$$\mathcal{L}_{\text{nll}} = \frac{1}{BG} \sum_{i=1}^B \sum_{g=1}^G \frac{1}{2} \left[\log \sigma_{ig}^2 + \frac{(y_{ig} - \mu_{ig})^2}{\sigma_{ig}^2} \right]. \quad (4.18)$$

Here B is the mini batch size and G is the number of genes. The term y_{ig} is the true response, μ_{ig} is the predicted mean, and σ_{ig}^2 is the predicted variance. In the implementation, the model predicts $\log \sigma_{ig}^2$ for numerical stability.

The model also uses a sample wise Pearson correlation term. For one sample i , the correlation over the selected genes is

$$\text{corr}(y_i, \mu_i) = \frac{\text{cov}(y_i, \mu_i)}{\text{std}(y_i) \text{std}(\mu_i)}, \quad (4.19)$$

where the covariance and standard deviations are computed over genes within sample i . The correlation loss is

$$\mathcal{L}_{\text{pcc}} = 1 - \frac{1}{B} \sum_{i=1}^B \text{corr}(y_i, \mu_i). \quad (4.20)$$

This form is used because the training objective is minimized. A larger Pearson correlation should improve the model, so the optimization uses $1 - \text{PCC}$ rather than PCC itself.

The main prediction objective is

$$\mathcal{L}_{\text{full}} = \mathcal{L}_{\text{nll}} + \lambda_{\text{pcc}} \mathcal{L}_{\text{pcc}}. \quad (4.21)$$

The likelihood term reduces prediction error while learning a gene wise uncertainty level. The PCC term encourages agreement in the shape of the response profile across genes.

A related CAGNN variant also adds a counterfactual drug loss. In that variant, one forward pass uses the true drug target vector and another uses a zero drug target vector. This extension is introduced to study how strongly the model depends on the drug signal, rather than to redefine the main benchmark. Let the second prediction loss be $\mathcal{L}_{\text{cf}}$. The hinge penalty is

$$\mathcal{L}_{\text{hinge}} = \max(0, \delta - (\mathcal{L}_{\text{cf}} - \mathcal{L}_{\text{full}})). \quad (4.22)$$

The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{full}} + \lambda_{\text{cf}} \mathcal{L}_{\text{hinge}}. \quad (4.23)$$

This term encourages the prediction with the real drug input to be better than the prediction with the zero drug input [31, 32, 33].

Training settings

The proposed model is trained on the paired perturbation data with the same split settings used for the baselines. The main benchmark, the ablation analysis, the uncertainty analysis, and the warm split data efficiency study all keep the same graph backbone, uncertainty output, target definition, and split design. Optimization uses Adam with a learning rate of 5×10^{-4} , mini batches, and 50 epochs. In the counterfactual variant, one forward pass uses the true drug target vector and another uses a zero drug vector.

Table 4.2 summarizes the training configuration used for CAGNN. The node readout is applied independently to each gene node. The main drug input is the target based drug vector.

Table 4.2: Training configuration of the proposed model.

Setting	Value
Optimizer	Adam
Learning rate	5×10^{-4}
Batch size	32
Training epochs	50
Hidden dimension	64
Graph attention layers	4
Attention heads	4
Dropout	0.2
Output	μ and $\log \sigma^2$
Pearson loss weight λ_{pcc}	5
Counterfactual weight λ_{cf}	1
Counterfactual margin δ	0.1

4.6 Evaluation Metrics

Split settings

Three split settings are used. Warm split uses a random sample split and evaluates performance on data that look similar to the training set. Cold drug split holds out test drugs from training and measures how well the model works on drugs it has not seen before. Cold cell split holds out test cell lines from training and measures how well the model works on cell lines it has not seen before. In this pipeline, the training set uses multiple matched controls for each treatment profile.

Average baselines

In addition to the learned models, this thesis also reports several simple average based baselines. These baselines do not train a neural network. Instead, they make predictions directly from averages computed on the training set, showing how much the learned models improve over very simple rules.

The simplest baseline is `Mean`, which is obtained by averaging all training response vectors gene by gene. This produces one fixed vector of length 978, and that same vector is used as the prediction for every test sample. As a result, `Mean` ignores both drug identity and cell identity and represents only the average response pattern observed in the training set.

$$\bar{y}_{\text{all}} = \text{mean}(Y_{\text{train}}), \quad \hat{y}_j = \bar{y}_{\text{all}} \text{ for every test sample } j. \quad (4.24)$$

When this baseline is evaluated with PCC, the same average vector is compared with each test sample separately. For every test sample, one sample wise PCC is computed with Eq. 4.19, and the final reported test PCC is the mean of these sample wise values across the full test set. Figure 4.8 illustrates this calculation process.

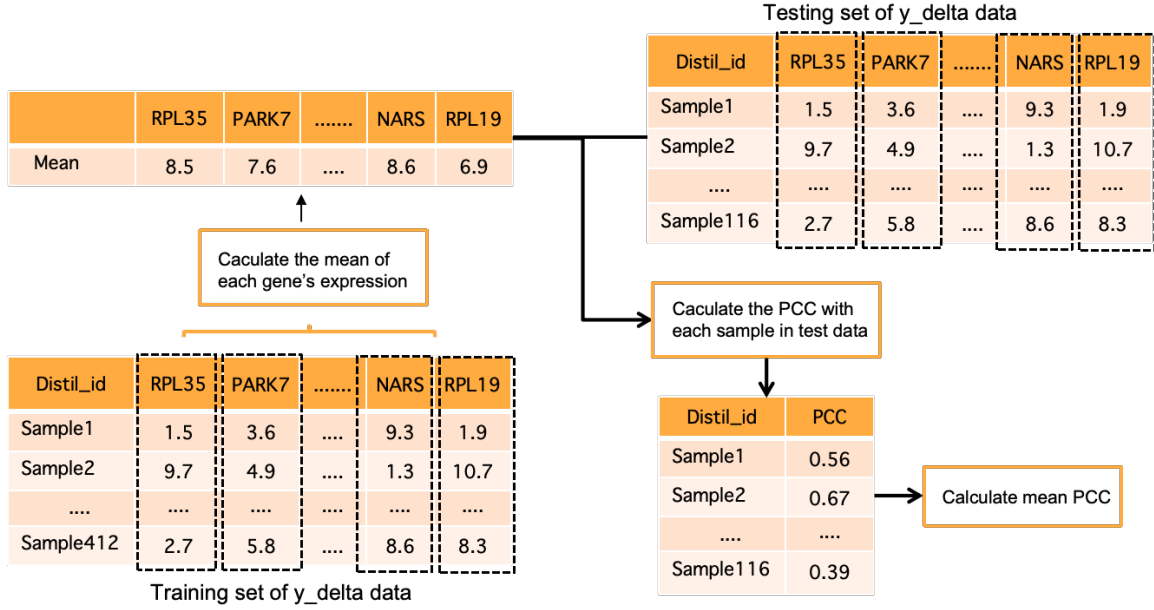


Figure 4.8: Illustration of how mean test PCC is computed for the `Mean` baseline. An average response vector is first estimated from the training set and then repeated as the prediction for every test sample. PCC is computed between each predicted row and the corresponding true response row using Eq. 4.19.

Another baseline is `mean_cell`. In this case, the training samples are grouped by cell line, and the response vectors within each group are averaged gene by gene. During testing, each sample is assigned the average vector of its corresponding cell line. This baseline uses cell identity, while still ignoring which drug is applied.

$$\bar{y}_{\text{cell}} = \text{mean}(\text{training responses from one cell line}), \quad \hat{y} = \bar{y}_{\text{cell}}. \quad (4.25)$$

Here, $\bar{y}_{\text{cell}}$ means the average response vector of the cell line to which the test sample belongs. When `mean_cell` is evaluated with PCC, the test samples are first grouped by cell line. For each cell line that appears in the training set, one average response vector is computed from the corresponding training samples. Each test sample then receives the average vector of its own cell line as prediction, and PCC is computed with Eq. 4.19. The final reported test PCC is the mean of these PCC values across the full test set. Figure 4.9 illustrates this calculation process.

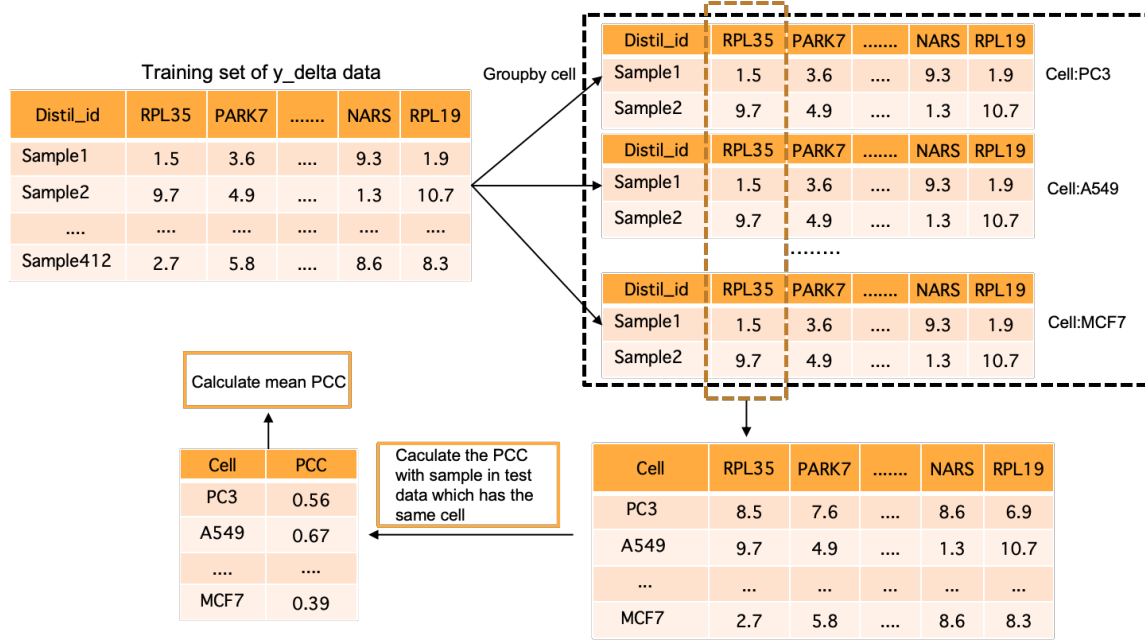


Figure 4.9: Illustration of how mean test PCC is computed for the `mean_cell` baseline. The training responses are first grouped by cell line, and one average response vector is computed for each cell. Each test sample then receives the average vector of its corresponding cell line. Sample wise PCC is computed between each predicted row and the corresponding true response row using Eq. 4.19, and the final test PCC is the mean of these sample wise PCC values.

`mean_drug` is computed in the same way as `mean_cell`, except that the grouping variable is the drug rather than the cell line. For each drug in the training set, the response vectors are averaged gene by gene, and each test sample receives the average vector of its drug.

$$\bar{y}_{\text{drug}} = \text{mean}(\text{training responses from one drug}), \quad \hat{y} = \bar{y}_{\text{drug}}. \quad (4.26)$$

Here, $\bar{y}_{\text{drug}}$ means the average response vector of the drug used in the test sample. Not every baseline is meaningful in every split. In the cold drug split, the test drugs are unseen during training, so a drug specific average cannot be computed for them, so `mean_drug` is omitted. In the cold cell split, the same logic applies to unseen cell lines, so `mean_cell` is omitted there. By contrast, `Mean` can always be evaluated because it does not depend on any specific drug or cell label.

Error metrics

This metric shows how close the predicted response values are to the observed response values. A lower MSE means that the predicted perturbation profile is closer to the true profile.

$$\text{MSE} = \text{mean}((y - \hat{y})^2). \quad (4.27)$$

MSE checks not only the direction of change but also the approximate size of the response. This matters when the strength of perturbation is important. However, MSE alone does not show whether the model captures the overall response pattern well. A model may not get the exact numbers right but still preserve the main biological structure of the profile. In this thesis, MSE follows the same squared error view used in the loss terms introduced above.

Correlation metrics

The main correlation metric is Pearson correlation coefficient (PCC). This metric compares the predicted and true response profiles within one sample and asks whether they follow a similar gene

wise pattern. A higher PCC means that genes predicted to increase or decrease tend to match the observed trend more closely.

Perturbation analysis often cares about whether the overall gene pattern is correct, but a model can still be informative if it identifies which genes move together and which genes change in opposite directions, even when the exact response size is not perfectly matched. So we can use PCC together with MSE to evaluate the final result. The PCC definition follows Eq. 4.19, and the related loss function is given in Eq. 4.20.

PCC values in this task should not be compared directly with accuracy values from tasks such as classification. Here the model must predict a full continuous response profile across 978 genes for each sample. A good PCC requires the model to recover the relative pattern across many genes, not just to make one correct decision. In addition, drug response data contain measurement noise, different cell states, incomplete drug target information, and many genes with only weak responses. For these reasons, even a modest PCC can still indicate that the predicted gene pattern follows the observed pattern to a useful degree.

The evaluation also reports regulation sign accuracy. This metric asks whether the model predicts the correct direction of regulation for each gene, for example, whether the gene goes up or down. This matters because the sign of change tells whether a gene is up regulated or down regulated after treatment.

$$\text{Acc}_{\text{sign}} = \frac{\text{number of genes with correct sign}}{\text{number of compared genes}}. \quad (4.28)$$

This score is computed for each sample and then averaged across the test set, checking whether the predicted direction of change is correct.

Precision metrics

The evaluation also reports top gene precision. This metric asks whether the model can recover the genes with the strongest positive or negative responses. In practice, this is measured from ranked predicted and true response vectors. The goal is not to treat every small change equally, but to test whether the model identifies the most strongly affected genes.

For one sample, positive precision at 20 is defined as

$$\text{Pos P@20} = \frac{\text{predicted top 20 up genes that also appear in true top 100 up genes}}{20}. \quad (4.29)$$

Negative precision at 20 is defined in the same way for the down regulated genes,

$$\text{Neg P@20} = \frac{\text{predicted top 20 down genes that also appear in true top 100 down genes}}{20}. \quad (4.30)$$

This metric is useful because later analysis often focuses on the strongest genes. These genes are more likely to be discussed as key pathway genes or as representative signals of the drug effect. A model with good top gene precision is better at ranking the strongest changed genes.

Data efficiency metrics

Beyond the standard split settings, this thesis also includes a drug cell pair data efficiency evaluation on the warm split. In this setting, the test set is kept fixed, while the amount of available training data is increased step by step under several fixed settings. This makes it possible to study how quickly different models improve as more training data becomes available.

This reduction is not done by randomly dropping samples from the whole training set. Instead, it is applied at the level of a drug cell pair. Here a drug cell pair means one specific drug tested in one specific cell line. For example, all repeated measurements of drug A in the A549 cell line form one pair, while the same drug A in MCF7 is counted as a different pair. If the same drug is measured several times in the same cell line, all of these repeated samples belong to the same pair.

Two simple data settings are used. In the one shot setting, only one training sample is kept for each drug cell pair. In the fraction based setting, only a fixed fraction of the available samples is kept for each pair. For example, if one pair has ten training samples, then a 30% setting keeps about three of them, while an 80% setting keeps about eight. The kept samples are chosen randomly within each pair without replacement. If a pair has very few samples, the procedure still keeps at least one sample when the setting is positive. The test set is not changed during this process, so all models are compared on the same evaluation data.

The reason for testing settings such as one shot, 30%, or 80% is that they represent different levels of data availability. One shot is an extreme low data setting. It tests whether the model can still learn when each drug cell pair is seen only once. Mid range settings such as 30% show whether the model improves steadily when more repeated measurements become available. Higher settings such as 80% show whether performance is already near its highest level or whether the model still benefits from more data. Together, these settings make it possible to compare not only final accuracy, but also how the models improve as more training data are added.

This design has a clear advantage. It prevents a few large drug cell pairs from dominating the reduced training set. If samples were removed at random, common pairs would still keep many examples, while rare pairs could disappear entirely. That would make the comparison less fair and less informative about data efficiency at the pair level. With the pair based design, each observed drug cell combination remains represented, and the comparison focuses on how many repeated samples for each pair are needed.

This design also has limitations. It does not test what happens when an entire drug cell pair is missing from training, because each observed pair still keeps at least one sample for any positive setting. For that reason, this experiment is best read as a repeated measurement data efficiency test, not as a new unseen pair test. But it is still relevant because drug response data are often limited and expensive to collect. A model that still works when only a few samples are available for each pair is useful in that setting.

5

Results

This chapter presents the main results for CAGNN and the baseline models under the warm, cold drug, and cold cell splits. The results are organized into error, correlation, top gene precision, direction of change, ablation, uncertainty, data efficiency, and case studies.

5.1 Evaluation framework

All experiments in this chapter use the L1000 landmark setting with 978 genes. For every sample, the prediction target is the response vector $y = x_{\text{trt}} - x_{\text{ctl}}$. All models use the matched control profile and the drug input. CAGNN also uses cell context information.

The evaluation compares three models. The DeepCOP style baseline, which uses a feed forward network. GSNN [17], which is used as the graph based baseline and Counterfactual Attention GNN (CAGNN). The split settings, pairing rules, and metric definitions are introduced by Evaluation Metrics section in methodology chapter. The same evaluation framework is used here to compare the models.

Most tables and figures in this chapter show the main comparison between CAGNN and the baseline models. Later sections then switch to additional analyses. The ablation section compares cell identity and control derived cell context within CAGNN. The uncertainty section studies the uncertainty output of the same CAGNN model. The data efficiency section studies warm split training budgets only. The case study section uses selected CAGNN examples to help explain the model behavior.

Table 5.1: Summary of the evaluation protocol used in the thesis.

Component	Setting
Dataset	LINCS L1000 landmark genes, 978 genes
Prediction target	Differential response $y = x_{\text{trt}} - x_{\text{ctl}}$
Models	DeepCOP baseline, GSNN, CAGNN
Splits	Warm, cold drug, cold cell
Primary metrics	Test MSE, sample wise test PCC, sign accuracy, Pos P@20, and Neg P@20
Top gene analysis	Pos P@20 and Neg P@20
Additional analyses	Ablation, uncertainty, data efficiency, and case studies
Attention analysis	Case study on specific drug

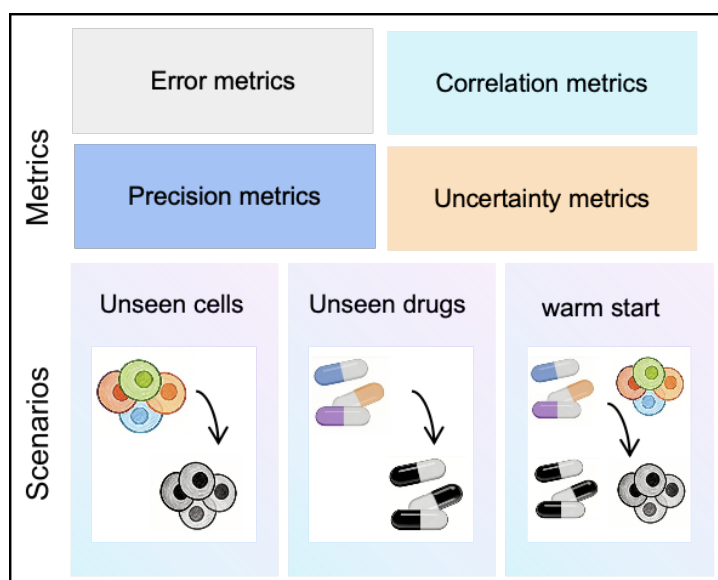


Figure 5.1: Simple view of the evaluation configuration used in this chapter.

Figure 5.1 shows the three split settings, the main metric groups, and the overall comparison logic. The later sections follow this same layout.

5.2 Error metric results

Error metrics measure how close the predicted values are to the observed values. The main error metric here is mean squared error (MSE), where a lower score means a better prediction. Table 5.2 and Figure 5.2 show the same ranking in table and plot form. CAGNN gives the lowest MSE in warm and cold drug, while DeepCOP is slightly best in cold cell. The clearest separation appears in cold drug, whereas cold cell shows a smaller gap and a different ranking.

Table 5.2: Test MSE under the three evaluation splits.

Model	Split	Test MSE
DeepCOP	Warm	0.8915
DeepCOP	Cold drug	1.1769
DeepCOP	Cold cell	0.8133
GSNN	Warm	0.9609
GSNN	Cold drug	1.0901
GSNN	Cold cell	0.8466
CAGNN	Warm	0.8182
CAGNN	Cold drug	0.9516
CAGNN	Cold cell	0.8384

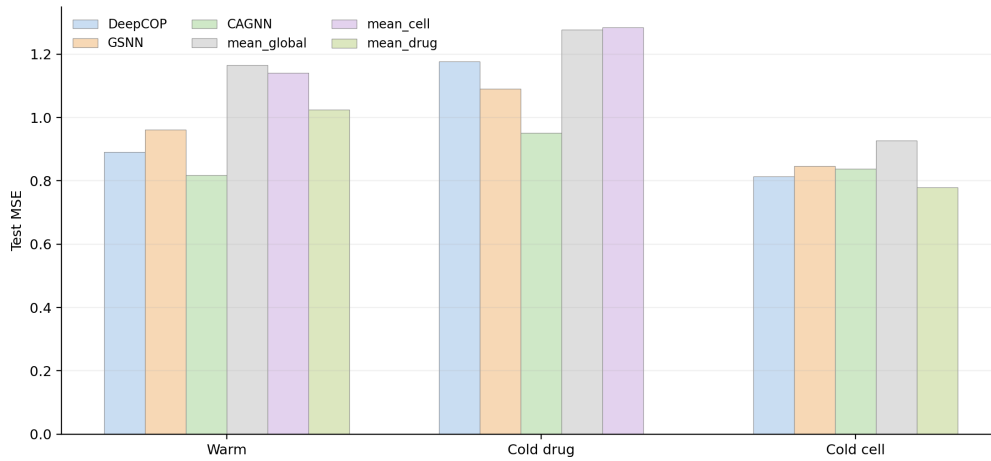


Figure 5.2: Test MSE in warm, cold drug, and cold cell for the learned models and the simple mean baselines. `mean_drug` is not shown in cold drug and `mean_cell` is not shown in cold cell because these baselines cannot be used there.

Scatter plots of y_{true} and y_{pred} show this ranking more clearly. Points close to the diagonal indicate smaller errors, while a wider cloud indicates larger deviations. In Figure 5.3, the CAGNN panels are tighter around the diagonal in warm and cold drug than the baseline panels. In cold cell, the point clouds become wider for all three models and the differences become smaller. This matches Table 5.2.

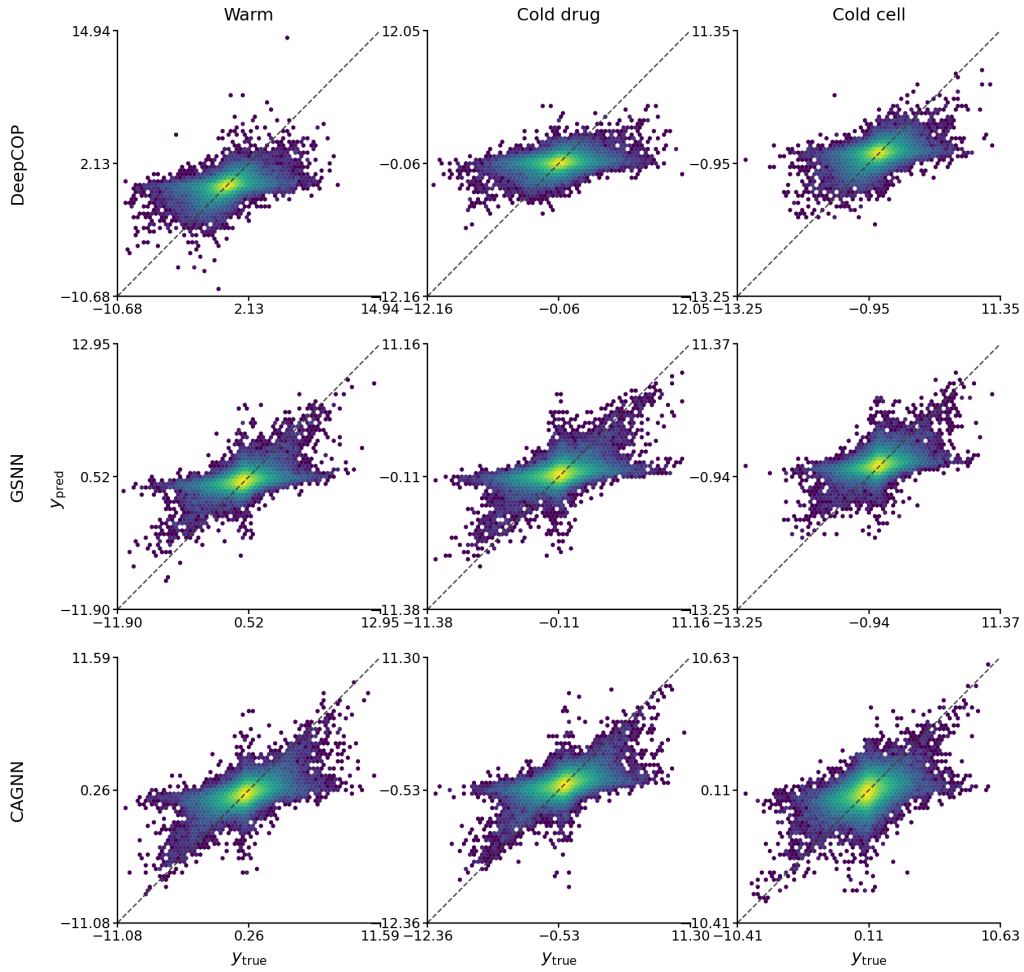


Figure 5.3: Gene level scatter plots for DeepCOP, GSNN, and CAGNN in the three splits.

5.3 Precision metric results

This section looks at whether the models recover the strongest gene level changes. This is measured by top gene precision, which emphasizes the genes with the largest positive and negative responses. For each sample, the genes are ranked in both the true response (y_{true}) and the predicted response (y_{pred}). Positive precision at 20, written as Pos P@20, measures how many of the 20 most up regulated genes in y_{pred} also appear among the top 100 most up regulated genes in y_{true} . Negative precision at 20, written as Neg P@20, is defined in the same way for the 20 most down regulated genes. The final score is the mean across all test samples. These metrics give another view of prediction quality that is closer to how perturbation results are often used in practice.

Table 5.3 and Figure 5.4 show the same result in table and plot form. CAGNN has the highest Pos P@20 and Neg P@20 in every split. The advantage is clear in warm and cold drug, and it remains visible in cold cell.

Table 5.3: Top gene precision results computed from the exported prediction files.

Model	Split	Pos P@20	Neg P@20
DeepCOP	Warm	0.5406	0.5402
DeepCOP	Cold drug	0.3892	0.4003
DeepCOP	Cold cell	0.4062	0.4129
GSNN	Warm	0.5179	0.4827
GSNN	Cold drug	0.5062	0.4855
GSNN	Cold cell	0.4779	0.4388
CAGNN	Warm	0.6180	0.6280
CAGNN	Cold drug	0.6035	0.5952
CAGNN	Cold cell	0.5344	0.4993

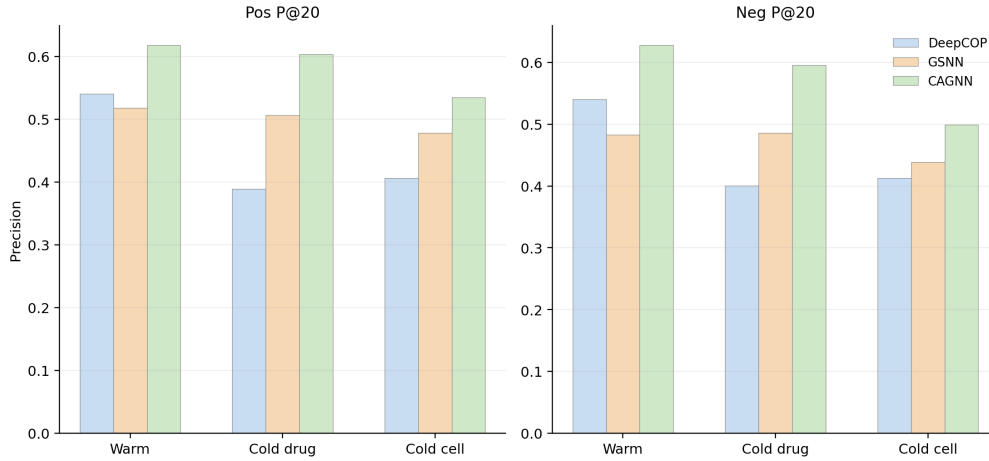


Figure 5.4: The left panel shows Pos P@20 and the right panel shows Neg P@20 for DeepCOP, GSNN, and CAGNN in the three splits.

5.4 Correlation metric results

The Pearson correlation coefficient (PCC) measures whether the predicted response follows the same overall gene wise pattern as the true response. A higher score means better agreement in shape for each sample.

Table 5.4 and Figure 5.5 show the same PCC results in table and plot form. CAGNN is highest in all three splits. The strongest baseline changes by split. DeepCOP is slightly stronger than GSNN in warm, while GSNN is clearly stronger in cold drug and cold cell. Another clear result is that PCC decreases for all methods from warm to cold cell. In Figure 5.5, the CAGNN bar is highest in each split and the cold cell bars are lower overall.

Table 5.4: Test PCC under the three evaluation splits.

Model	Split	Test PCC
DeepCOP	Warm	0.4671
DeepCOP	Cold drug	0.3104
DeepCOP	Cold cell	0.3457
GSNN	Warm	0.4609
GSNN	Cold drug	0.4556
GSNN	Cold cell	0.4237
CAGNN	Warm	0.5605
CAGNN	Cold drug	0.5424
CAGNN	Cold cell	0.4783

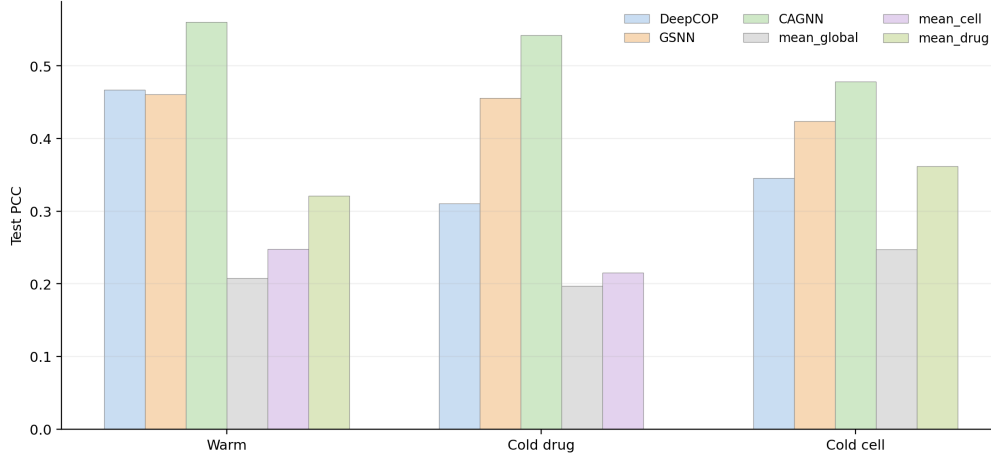


Figure 5.5: Test PCC in warm, cold drug, and cold cell for the learned models and the simple mean baselines. mean_drug is not shown in cold drug and mean_cell is not shown in cold cell because these baselines cannot be used there.

Regulation sign results

This metric is used to examine agreement in the direction of change at the gene level. The definition of regulation sign accuracy is introduced in the section Evaluation Metrics. Here, the focus is on how the models compare under that metric. The score does not use the response size. It only checks whether the predicted direction of change is correct.

Table 5.5 shows the sign agreement results under the three splits. The winner changes with the split. DeepCOP is highest in warm, CAGNN is highest in cold drug, and GSNN is highest in cold cell. This means the sign metric gives a more mixed ranking than PCC. It also shows that predicting only the direction of change is not exactly the same as predicting the full response shape.

Table 5.5: Regulation sign accuracy under the three evaluation splits.

Model	Split	Sign accuracy
DeepCOP	Warm	0.6582
DeepCOP	Cold drug	0.5942
DeepCOP	Cold cell	0.5859
GSNN	Warm	0.6086
GSNN	Cold drug	0.6038
GSNN	Cold cell	0.6084
CAGNN	Warm	0.6496
CAGNN	Cold drug	0.6415
CAGNN	Cold cell	0.5801

Across the three splits, the lowest PCC values appear in cold cell for all models. The ranking by PCC is also more stable than the ranking by sign accuracy, since CAGNN remains highest in all three splits. The margin is largest in warm and cold drug and smaller in cold cell.

Gene distribution analysis

Figure 5.6 gives a gene level view of model output. The figure has three rows, one for each split, and each row shows the distribution of x_{deg} for twelve genes with strong true response.

The result is consistent with the summary metrics. In warm and cold drug, the CAGNN distributions are usually closest to the ground truth distributions across the selected genes. In cold cell, the agreement becomes weaker for all models, but CAGNN still tracks the ground truth more closely than the two baselines in most shown genes.

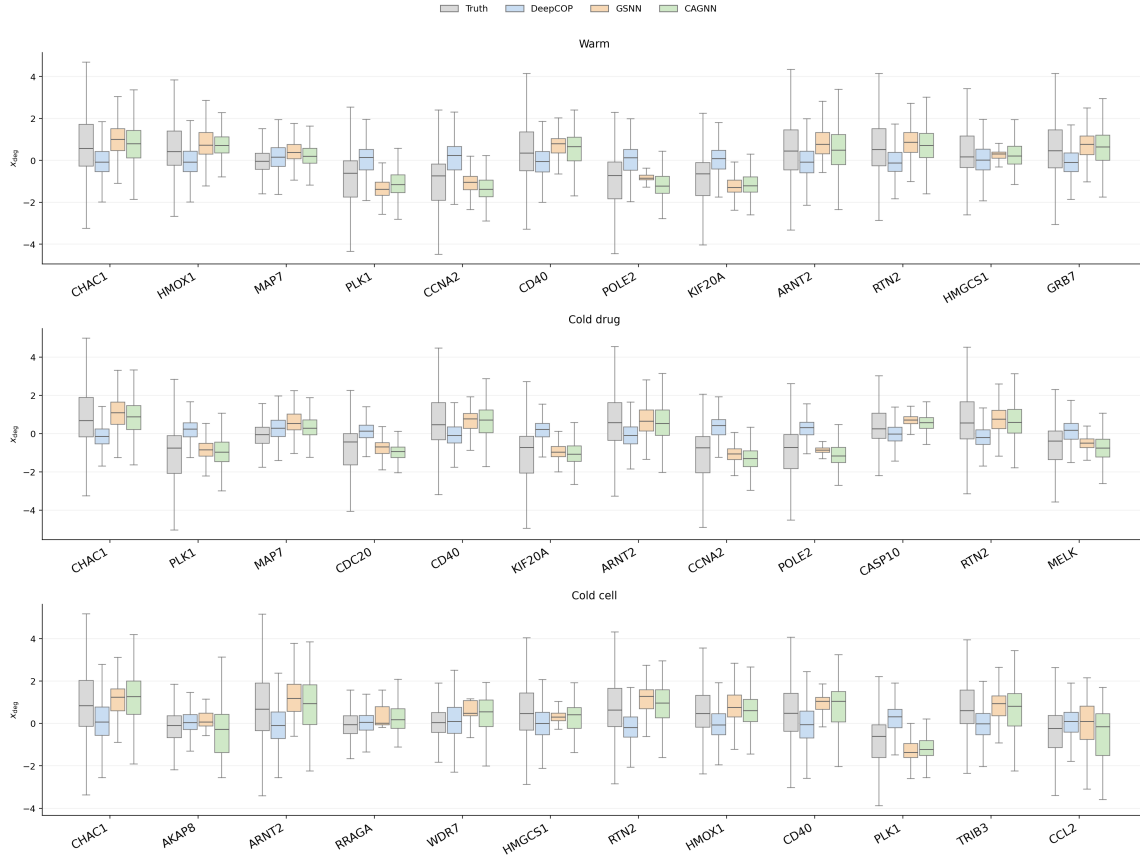


Figure 5.6: Gene level distribution in the three splits. The figure has three rows, one for each split. Each row shows x_{deg} for twelve genes with strong true response. Ground truth, DeepCOP, GSNN, and CAGNN are shown for each gene.

Ablation analysis

This ablation study is separate from the main benchmark comparison. It compares two ways of providing cell information to the proposed model. One variant uses only a cell identity embedding. The other uses a cell context derived from the matched control profile.

Table 5.6 shows the results. Here Δ means cell context minus cell identity, so a negative ΔMSE and a positive ΔPCC both indicate improvement. The cell context variant improves both metrics in all three splits. The changes are modest in warm and cold drug, but much larger in cold cell. This means that the benefit of cell context appears in every split and is strongest when the test cell line is unseen.

Table 5.6: Comparison of cell information variants in CAGNN. The table reports test MSE and test PCC for cell identity and cell context, together with their differences. Here Δ means cell context minus cell identity, so negative ΔMSE and positive ΔPCC indicate improvement.

Split	Identity MSE	Identity PCC	Context MSE	Context PCC	ΔMSE	ΔPCC
Warm	0.8567	0.5364	0.8218	0.5601	-0.0349	+0.0237
Cold drug	0.9891	0.5174	0.9542	0.5401	-0.0349	+0.0227
Cold cell	1.6983	0.3709	0.8393	0.4778	-0.8590	+0.1069

5.5 Uncertainty analysis

In CAGNN, the model predicts both a mean response and a gene level variance term. This section asks whether that uncertainty output behaves like a reliability signal. In this section, μ denotes the

predicted mean response, σ^2 denotes the predicted variance, and σ is the standard deviation derived from that variance. Figure 5.7 shows three views of this output, namely sample level uncertainty, uncertainty versus error, and grouped uncertainty analysis for warm, cold drug, and cold cell.

Sample level uncertainty

Each test sample contains a full vector of predicted gene responses, and each gene also has its own predicted uncertainty. To obtain one simple uncertainty value for the whole sample, the average predicted standard deviation across all genes is used. The left panel of Figure 5.7 compares this quantity across the three splits. Warm and cold drug have similar distributions, while cold cell is shifted slightly upward and shows a somewhat broader upper range.

Uncertainty and MSE

To check whether the model becomes more uncertain when it makes larger mistakes, each test sample is plotted by its sample level uncertainty and its sample wise MSE. In all three splits, the point clouds show a positive relationship. The trend is strongest in cold cell, moderate in warm, and weaker in cold drug. When reading the middle upper panel of Figure 5.7, the key pattern is that higher uncertainty values are usually associated with higher MSE.

Uncertainty and PCC

A related analysis compares the same uncertainty value with sample wise PCC. This gives a different view from MSE. In Figure 5.7, only cold cell shows a clearly visible negative relationship. In warm and cold drug, the relationship between uncertainty and PCC is much weaker. This means that the uncertainty output follows absolute error more clearly than sample wise correlation.

Uncertainty groups analysis

The same pattern can be shown more directly by grouping test samples into low, medium, and high uncertainty groups. The groups are formed by sorting the samples according to sample level σ and splitting them into three equal sized parts. The average MSE and PCC are then calculated inside each group. In the right panel of Figure 5.7, the main pattern is clear for MSE in all three splits, since the high uncertainty group always has the largest mean error. For PCC, the separation is visible mainly in cold cell and much weaker in warm and cold drug.

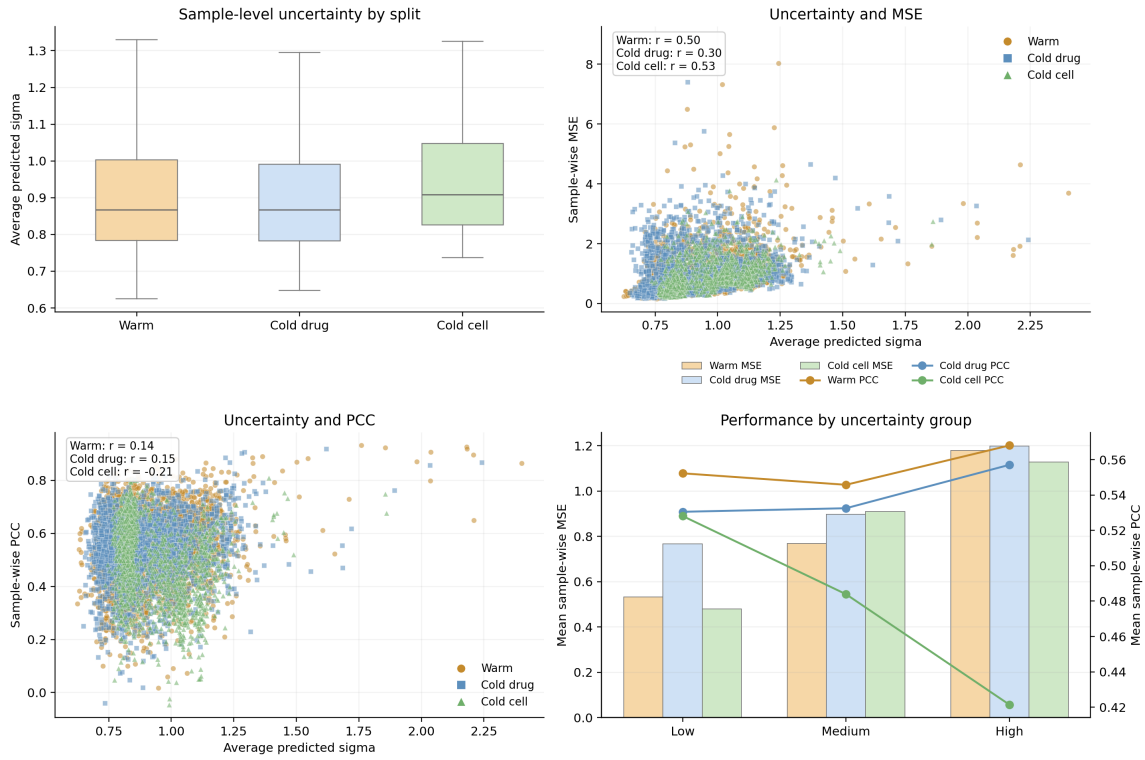


Figure 5.7: Uncertainty summary in warm, cold drug, and cold cell, based on the test set. The left panel shows sample level uncertainty in the three splits. The middle panels compare uncertainty with MSE and PCC. The right panel shows MSE and PCC for low, medium, and high uncertainty groups in each split.

Taken together, Figure 5.7 shows three main points. Cold cell is slightly more uncertain overall. Uncertainty is more clearly related to MSE than to PCC. High uncertainty groups also have larger MSE in all three splits.

Coverage of prediction bands

Under a Gaussian interpretation, the band $\mu \pm 2\sigma$ can be read as an approximate 95% prediction interval. To check this more directly, the empirical coverage is calculated on the full test set as the fraction of target genes whose true values fall inside $\mu \pm 2\sigma$. Table 5.7 shows that the coverage is close to 95% in all three splits.

Table 5.7: Coverage of the prediction band $\mu \pm 2\sigma$ on the full test set.

	Warm	Cold drug	Cold cell
Coverage	95.96%	94.00%	96.30%

Gene uncertainty

The uncertainty output can also be studied at the gene level instead of the sample level. For each gene, the predicted standard deviation is averaged across all test samples. This makes it possible to identify genes that are repeatedly assigned higher uncertainty and to compare gene level uncertainty with gene level prediction error. Figure 5.8 shows this view.

The left panel shows the genes with the highest mean predicted σ . The right panel compares mean gene level σ with gene level MSE. The main pattern is again positive, since genes with larger mean uncertainty also tend to have larger gene level MSE.

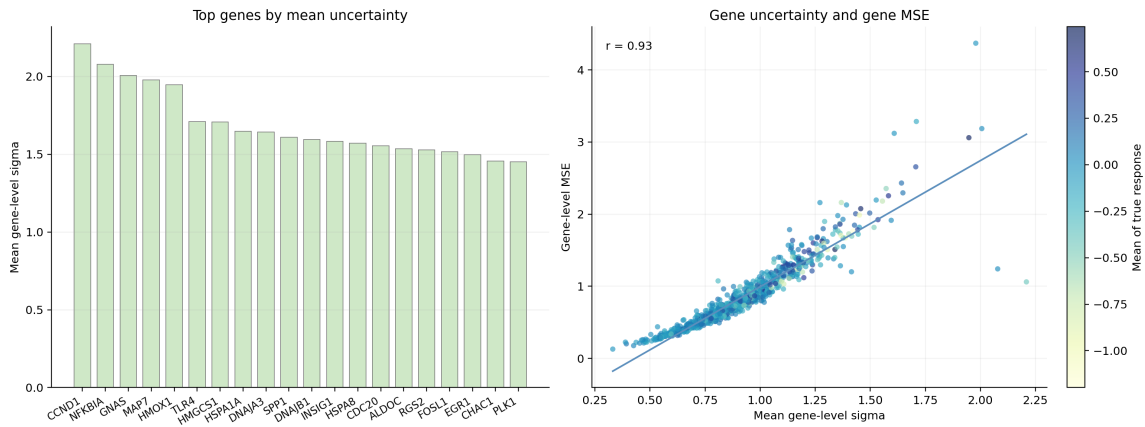


Figure 5.8: The left panel shows the top 20 genes with the highest mean predicted standard deviation on the test set. The right panel compares mean gene level σ with gene level MSE for all target genes on the test set.

Case studies with mean and uncertainty

The uncertainty output can also be inspected in individual examples. In Figure 5.9, each panel shows one test sample. The plot includes the true response, the predicted mean, and the prediction band $\mu \pm 2\sigma$. Table 5.7 already shows that this band gives close to 95% coverage on the full test set. To keep the figure readable, each panel shows only the twelve genes with the largest absolute true response in that sample. The selection is based on y_{true} , not on y_{pred} . For each split, the left panel shows a lower uncertainty sample with a better fit, while the right panel shows a higher uncertainty sample with a larger error. These displayed cases are only visual examples, so their local coverage can differ from the full test set coverage.

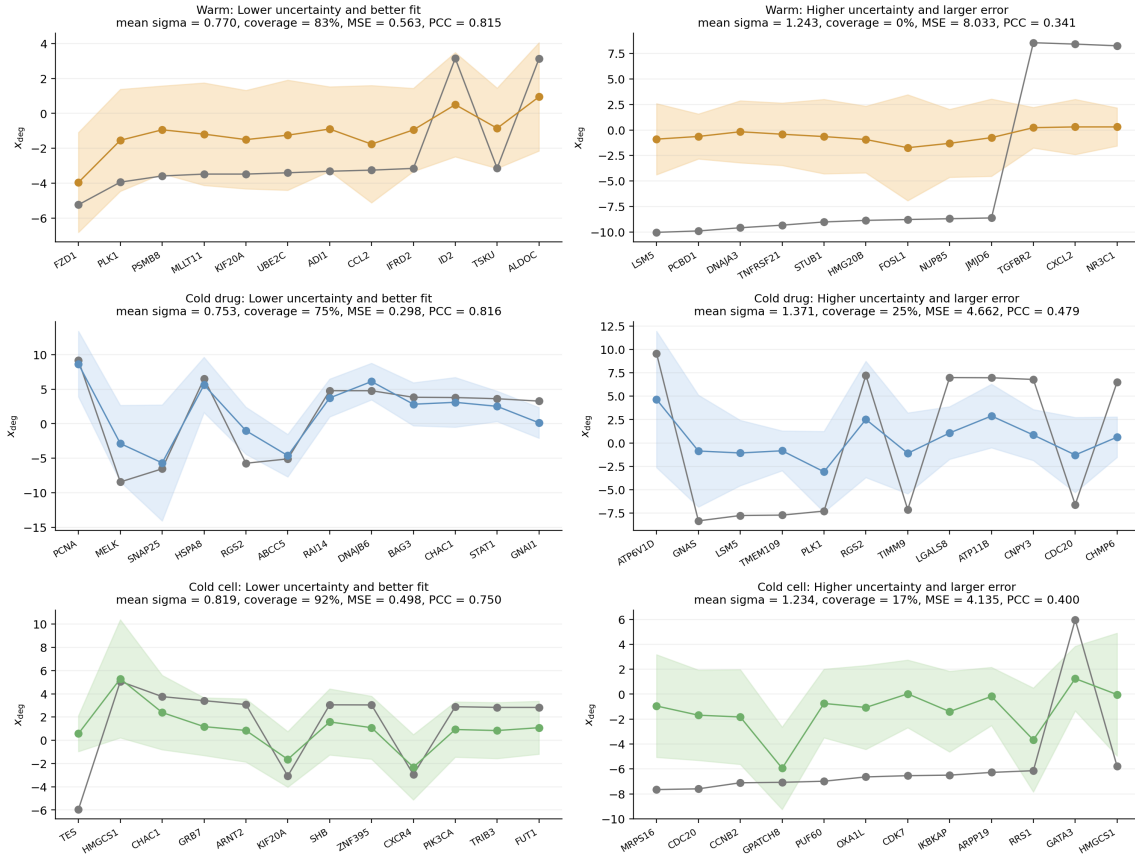


Figure 5.9: Case studies with uncertainty bands in warm, cold drug, and cold cell. In each split, the left panel shows a lower uncertainty sample with a better fit, and the right panel shows a higher uncertainty sample with a larger error. Each panel uses the twelve genes with the largest absolute true response in that sample. The plots show the true response, the predicted mean, and the prediction band $\mu \pm 2\sigma$.

5.6 Data efficiency evaluation

This section examines how model performance changes as the amount of training data increases in the warm split. It is a separate budget analysis rather than another three split benchmark table. The detailed setup is introduced in the methodology chapter. Figure 5.10 shows how the models improve as more training samples are available.

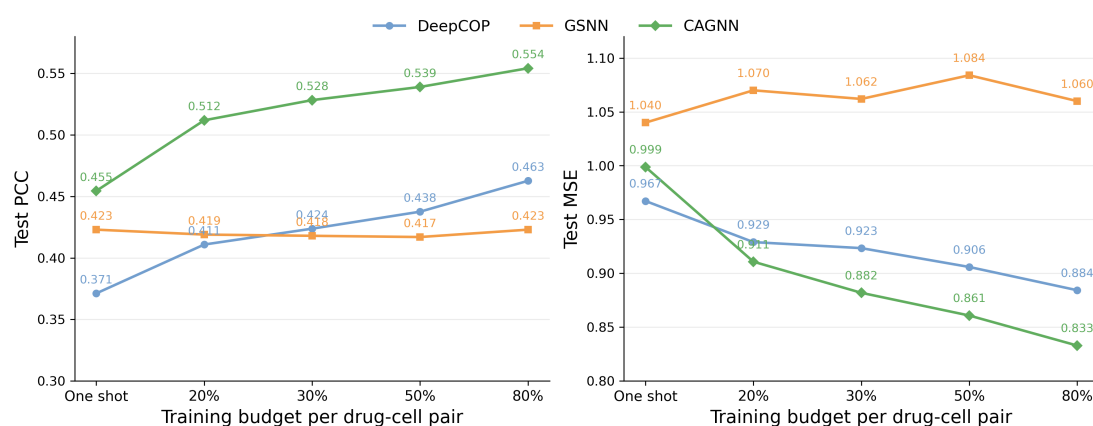


Figure 5.10: Warm split data efficiency results. The left panel shows test PCC and the right panel shows test MSE for DeepCOP, GSNN, and CAGNN across the displayed training budgets. In the updated DeepCOP budget runs, DeepCOP also improves clearly as more training data are used, while GSNN changes less across the same budgets.

Figure 5.10 shows that CAGNN remains strongest on PCC across the displayed warm split budgets, and its advantage is already visible at smaller budgets. In the updated DeepCOP budget runs, DeepCOP also improves steadily as more data are added, with higher PCC and lower MSE across the shown budgets. GSNN changes less over the same range.

5.7 Interpretation and case studies

Average metrics show overall performance, but they do not show how the model uses the graph in one specific example. This section looks at selected cases from CAGNN. Here attention means the weight that the model gives to one graph edge during message passing. A larger weight means that the model uses that edge more strongly in the final prediction. These weights are not treated as proof of a real biological mechanism. The analysis instead asks two simple questions. Are the highly weighted edges easier to understand when the prediction is accurate, and does the model keep giving high weights to similar edges across different settings.

Table 5.8 gives short explanations of the main biology terms used in this section. These short notes are only meant to help reading. They do not replace the full biological background of each gene or drug class.

Table 5.8: Short explanations of selected biology terms used in the attention analysis and case studies.

Term	Simple meaning
SB-939	A drug in the case study. It is reported as an HDAC inhibitor.
mifobate	A drug in the case study. It is reported to act on PPARG.
alvocidib	A drug in the case study. It is reported as a CDK inhibitor.
HDAC inhibitor	A drug class that acts on proteins that help control which genes are turned on or off.
PPARG	A regulator related to cell state and basic cell activity.
CDK inhibitor	A drug class that acts on proteins involved in cell division.
BCL2 family proteins	Proteins involved in whether a cell survives or dies.
TP53	A broad stress and damage response regulator. It is often active when the cell is under pressure or has been damaged.
NR3C1	A regulator related to stress response and cell state.

For the cold drug split, the drugs are ranked by prediction quality. Only drugs with at least ten test samples are kept, so that the comparison is based on a stable group of examples.

Figure 5.11 and Figure 5.12 show where the model places the largest edge weights for the 20 best and 20 worst predicted drugs. Each figure highlights the strongest connections. This gives a view of whether the model keeps its focus on a smaller set of edges or spreads the weight across many edges.

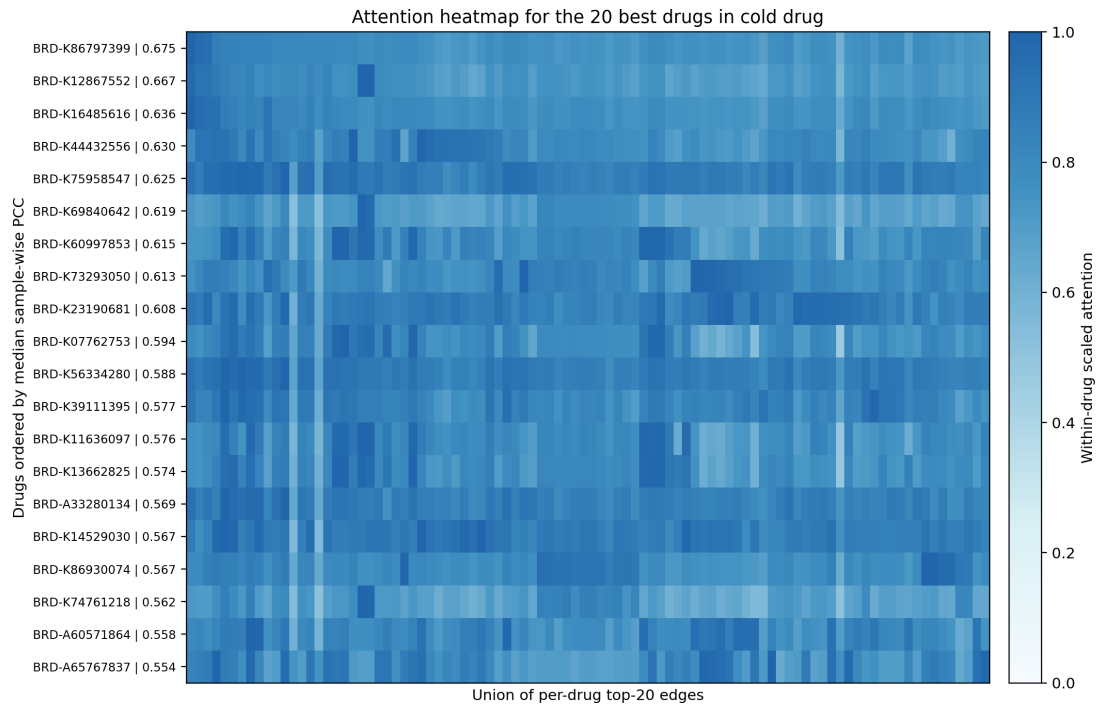


Figure 5.11: Attention heatmap for the top 20 best drugs in the cold drug split. For each drug, the heatmap shows the strongest edges between different genes. The values are scaled within each drug. Darker blue regions show where the model puts more attention.

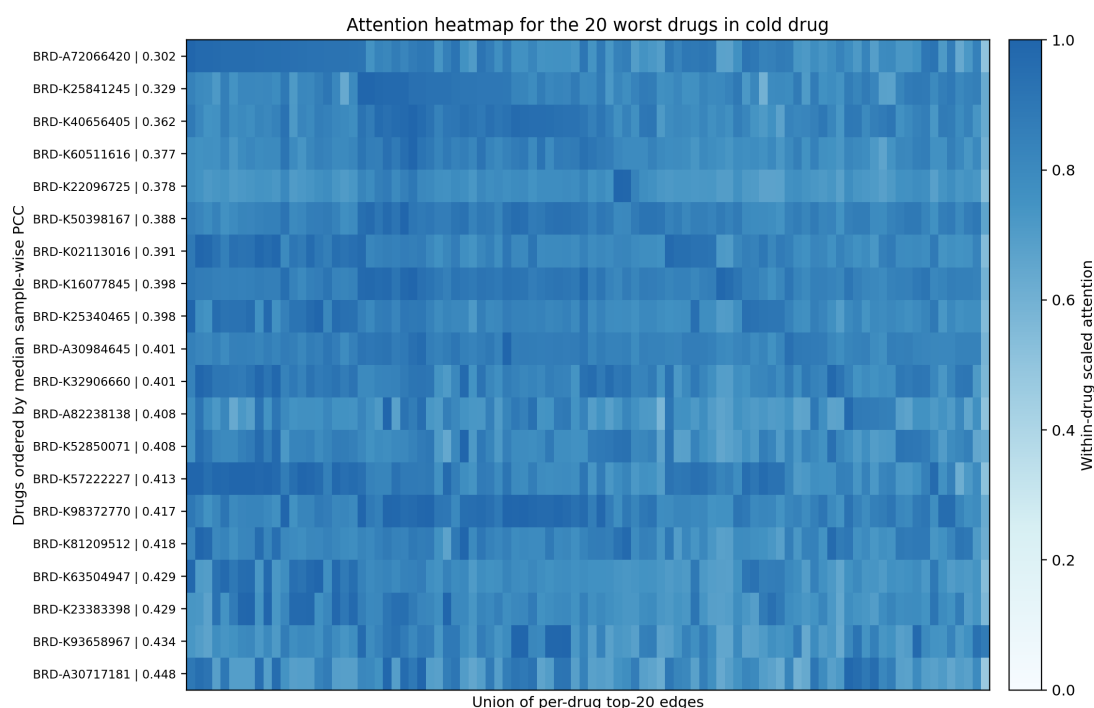


Figure 5.12: Attention heatmap for the top 20 worst drugs in the cold drug split. For each drug, the heatmap shows the strongest edges between different genes. The values are adjusted within each drug. Darker blue regions show where the model puts more attention.

These two heatmaps give an overview before looking at individual drugs. In the best predicted cases, darker values often appear in a smaller and more repeated set of edges. In the worst predicted cases, the weights are spread more widely. But weak predictions can still contain large edge weights. For that reason, the heatmaps alone are not enough, and the edge lists in Table 5.9 are needed for closer reading.

In the updated cold drug ranking, the best drug with enough test samples is SB-939, labeled as BRD-K86797399. This compound is reported as an HDAC inhibitor. The weakest prediction is mifobate, labeled as BRD-A72066420, which is reported to act on PPARG. In the SB-939 case, the top edges include NFE2L2→TXNRD1 and STAT3→PROS1. These edges involve genes related to broad cell responses. This makes the result easier to relate to downstream cell response genes.

In the mifobate case, the top edges include HIF1A→KDM3A and CDK1→RNMT. These edges mainly involve genes such as HIF1A, CDK1, and MYC, which are related to many common cell processes. Although these are valid graph edges, they are harder to connect to the reported target of the drug. In this example, large edge weights are less helpful for explanation.

Table 5.9: Top high attention edges for the best and worst cold drug examples. The left three columns show the best cold drug, BRD-K86797399. The right three columns show the worst cold drug, BRD-A72066420. The type column indicates whether the prior graph marks the edge as stimulation or inhibition. Attention values come from the last attention layer.

Best cold drug edge	Type	Attention	Worst cold drug edge	Type	Attention
NFE2L2→TXNRD1	stimulation	0.6672	HIF1A→KDM3A	stimulation	0.6995
STAT3→PROS1	stimulation	0.6364	CDK1→RNMT	stimulation	0.6812
STAT3→CSR1	stimulation	0.6325	CDK1→USP1	stimulation	0.6787
FAS→DAXX	stimulation	0.5709	MYC→ZNF451	stimulation	0.6751
GATA2→MAPKAPK3	stimulation	0.5646	HIF1A→ALDOA	stimulation	0.6710
RB1→TP53BP2	stimulation	0.5635	MYC→PAICS	stimulation	0.6695
E2F2→EED	stimulation	0.5626	MYC→RFC2	stimulation	0.6677
HMGA2→IGF2BP2	stimulation	0.5601	MYC→FPGS	stimulation	0.6669
CEBPA→HSD17B11	stimulation	0.5596	CDK1→ORC1	stimulation	0.6664
USP7→SKP1	inhibition	0.5567	MYC→NIT1	stimulation	0.6639

Table 5.9 makes this contrast clearer. The main difference is not simply the size of the edge weights. The weak case can even have larger values. The more meaningful difference is whether the model keeps giving high weights to a small set of edges that can be described clearly. This means that attention should be read together with prediction accuracy, not on its own.

The cold cell test asks a different question. Instead of comparing different drugs, it asks whether the same drug keeps a readable set of high weight edges when the cell line is unseen. Figure 5.13 shows the top 20 best drugs in this setting.

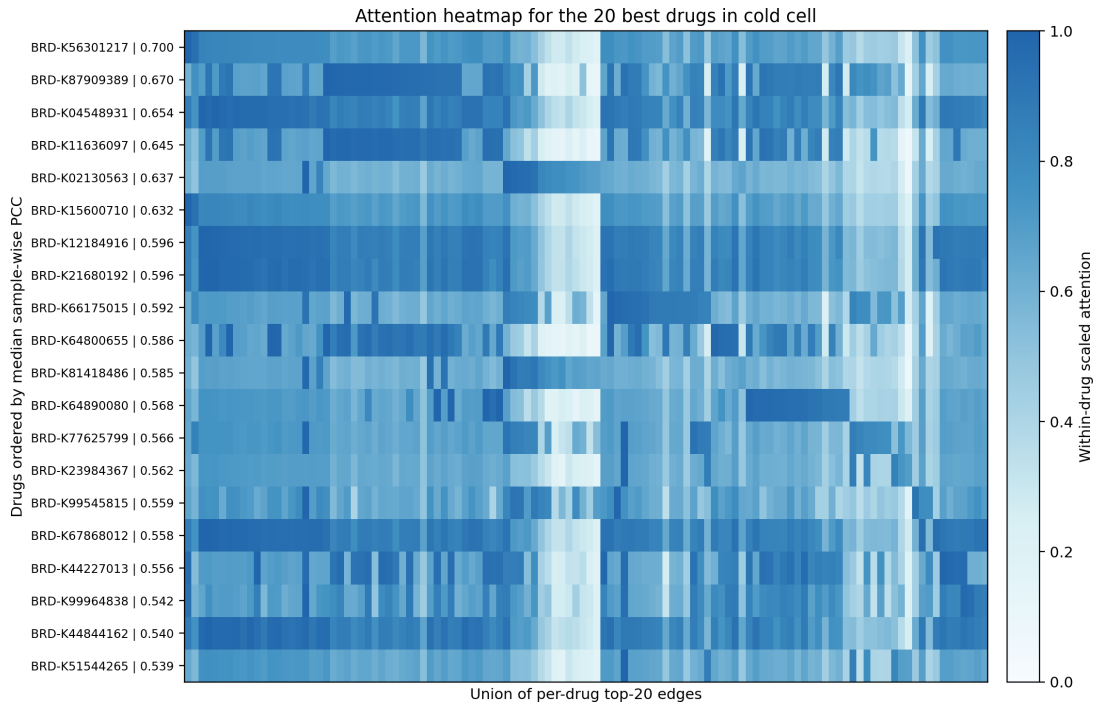


Figure 5.13: Attention heatmap for the top 20 best drugs in the cold cell split. For each drug, the heatmap shows the strongest edges between different genes. The values are scaled within each drug. Darker blue regions show where the model puts more attention.

Figure 5.13 shows that several well predicted cold cell drugs still produce clear edge weight patterns even when the cell line is unseen. Some sets of edges reappear across drugs. However, the heatmap

alone cannot show whether one specific drug keeps giving high weights to similar edges across warm and cold cell. That question needs a direct comparison of the same drug in both settings.

Table 5.10 compares the same drug, alvocidib labelled as BRD-K87909389, in warm and cold cell. This drug is reported as a CDK inhibitor. It is an appropriate example because the model predicts it well in both settings, with an accuracy of about 0.669 in warm and 0.670 in cold cell.

The table shows a shift in the highly weighted edges. In the warm setting, several top edges involve genes related to cell division, such as RB1 and E2F2. In the cold cell setting, more top edges involve genes related to broader stress and damage response, such as TP53 and NR3C1. This means that when the cell line is unseen, the model gives more weight to broad response genes than to the more specific genes seen in the warm setting.

Table 5.10: Top high attention edges for one shared drug in warm and cold cell. The same drug, BRD-K87909389, is shown in both settings. The type column indicates whether the prior graph marks the edge as stimulation or inhibition. Attention values come from the last attention layer.

Warm edge	Type	Attention	Cold cell edge	Type	Attention
RB1→TP53BP2	stimulation	0.8077	NR3C1→TIMELESS	stimulation	0.8136
HMGA2→IGF2BP2	stimulation	0.8006	TP53→ASCC3	stimulation	0.8044
GATA2→MAPKAPK3	stimulation	0.7999	TP53→CRYZ	inhibition	0.7978
ELAVL1→TRIB3	stimulation	0.7916	ABL1→CRKL	stimulation	0.7975
ELAVL1→ADAM10	stimulation	0.7899	TP53→P4HA2	stimulation	0.7950
E2F2→EED	stimulation	0.7892	TP53→REEP5	inhibition	0.7947
SMAD3→BAMBI	stimulation	0.7846	PLK1→SMC1A	inhibition	0.7947
GATA3→HAT1	stimulation	0.7845	ABL1→SORBS3	stimulation	0.7935
PAK1→PGM1	stimulation	0.7836	TP53→DUSP11	stimulation	0.7926
GATA3→BLCAP	stimulation	0.7832	TP53→CCNB2	inhibition	0.7919
GATA3→ZW10	stimulation	0.7824	TP53→ITFG1	stimulation	0.7857
GATA3→VGLL4	stimulation	0.7818	CHEK1→TLK2	inhibition	0.7794
HIF1A→SLC11A2	stimulation	0.7817	PLK1→ZMYM2	inhibition	0.7737
HIF1A→ALDOA	stimulation	0.7780	HIF1A→BNIP3	stimulation	0.7726
HIF1A→BNIP3	stimulation	0.7772	NR3C1→CEBPD	stimulation	0.7639
HIF1A→KDM3A	stimulation	0.7770	ELAVL1→ADAM10	stimulation	0.7629
TFDP1→NRIP1	stimulation	0.7766	MYC→PSMG1	stimulation	0.7614
HIF1A→S100A4	stimulation	0.7699	ELAVL1→TRIB3	stimulation	0.7596
MYC→PAICS	stimulation	0.7514	MYC→UBE2C	stimulation	0.7587
MYC→NIT1	stimulation	0.7512	HIF1A→KDM3A	stimulation	0.7583

At the same time, this comparison also shows a limitation. In cold cell, the model does not keep giving high weights to the same specific cell division related edges seen in warm. It still highlights some readable edges, but the result is less specific. This shows that unseen cell lines remain difficult, even when the overall prediction score is good.

Table 5.11 gives a second shared drug example, BRD-K56301217. The known targets of this drug are BCL2 family proteins.

The table shows a different result from the alvocidib example. The two highest ranked edges are BAX→VDAC1 and BECN1→PIK3R4. These two edges stay at the top in both warm and cold cell. Several other edges also overlap between the two settings. This means that the model keeps giving high weights to similar edges even when the cell background changes.

This consistency is useful when comparing the two settings. In the alvocidib example, the set of high weight edges became broader in cold cell. Here the result is more stable across settings. This shows that the model can keep giving high weights to a similar set of edges across different cell backgrounds for some drugs.

Table 5.11: Top high attention edges for a second shared drug in warm and cold cell. The drug is BRD-K56301217. Its known target genes are BCL2, BCL2L1, and BCL2L2. The type column indicates whether the prior graph marks the edge as stimulation or inhibition. Attention values come from the last attention layer.

Warm edge	Type	Attention	Cold cell edge	Type	Attention
BECN1→PIK3R4	stimulation	0.7320	BAX→VDAC1	stimulation	0.6833
BAX→VDAC1	stimulation	0.7176	BECN1→PIK3R4	stimulation	0.6426
RB1→TP53BP2	stimulation	0.5730	SRC→MAPKAPK5	stimulation	0.5716
E2F2→EED	stimulation	0.5427	GATA3→BLCAP	stimulation	0.5644
GATA2→MAPKAPK3	stimulation	0.5420	E2F2→EED	stimulation	0.5619
PCNA→POLD4	stimulation	0.5379	GATA2→MAPKAPK3	stimulation	0.5584
HMGA2→IGF2BP2	stimulation	0.5294	GATA3→VGLL4	stimulation	0.5558
HIF1A→KDM3A	stimulation	0.5283	SRC→CLTC	stimulation	0.5547
GATA3→ZW10	stimulation	0.5272	EGR1→EAPP	stimulation	0.5513
HIF1A→BNIP3	stimulation	0.5270	CEBPA→HSD17B11	stimulation	0.5474
CEBPA→HSD17B11	stimulation	0.5248	MYLK→MYL9	stimulation	0.5468
NFE2L2→TXNRD1	stimulation	0.5248	SRC→DNM1	stimulation	0.5460
PHKG2→PYGL	stimulation	0.5238	GATA3→ZW10	stimulation	0.5433
CFLAR→ATG3	inhibition	0.5236	GATA3→HAT1	stimulation	0.5410
GATA3→HAT1	stimulation	0.5207	CXCR4→GNAI2	stimulation	0.5409
SRC→MAPKAPK5	stimulation	0.5186	KIF5C→TRAK2	stimulation	0.5406
GATA3→VGLL4	stimulation	0.5186	SRC→MPZL1	stimulation	0.5404
NR3C1→TIMELESS	stimulation	0.5181	STAT3→CSR1	stimulation	0.5357
CSNK2A2→NOL3	stimulation	0.5180	STXBP1→STX1A	stimulation	0.5345
EGR1→DUSP4	stimulation	0.5175	NFE2L2→TXNRD1	stimulation	0.5316

Overall, these case studies show that attention can be helpful. When the prediction is strong, the highly weighted edges are often easier to connect to known drug targets or to a later response that looks reasonable. When the prediction is weak, large edge weights can still appear, but they are harder to summarize in a meaningful way. The warm and cold cell comparisons also show two possible results. For some drugs, the model shifts toward broader response genes in unseen cells. For other drugs, it keeps giving high weights to a more similar set of edges. This means that attention should be interpreted together with prediction quality and prior biological knowledge.

6

Discussion

6.1 Method

CAGNN was designed for drug effects that spread through the gene network. Known drug targets and a biological interaction graph provide structure without using a standard feed forward network. The main CAGNN comparison in this thesis combines a graph attention backbone, matched control expression, drug target input, control derived cell context, and gene wise uncertainty output. This main comparison is the basis for the three split comparisons with DeepCOP and GSNN. Separate parts of the thesis then examine additional questions. The ablation analysis compares cell identity with cell context inside the same backbone. The uncertainty analysis studies the uncertainty output of the same model. The warm split data efficiency analysis studies how performance changes when the training budget is reduced.

The baseline comparison serves two different purposes. DeepCOP shows what can be achieved without a graph, while GSNN provides a graph based baseline with a different message passing rule from CAGNN. The ablation analysis adds another comparison by testing how cell information should enter the model. Replacing cell identity with control derived cell context improves both MSE and PCC in all three splits, and the gain is much larger in cold cell. This helps explain where the task is hardest. In warm and cold drug, the model still sees familiar cell states during training, so a cell label already carries some useful information. In cold cell, this advantage becomes much weaker because the test cell line was never seen before. In that setting, the measured control profile becomes more valuable than the cell name, since it reflects the actual starting state of the sample.

6.2 Result

The main result is that known molecular structure improves prediction quality, although the size of the improvement depends on the test setting. In the three split comparison, CAGNN gives the most consistent strong results across the main metrics. It is best on PCC and top gene precision in all three splits, and it also gives the lowest MSE in warm and cold drug. In cold cell, DeepCOP gives a slightly lower MSE, but CAGNN remains stronger on PCC and top gene precision. This is important because drug response prediction is not a single number problem. A useful model should reduce numerical error, preserve the overall gene pattern, and recover the strongest changed genes.

The split comparison also shows a clear limit. On PCC and top gene precision, cold cell remains the hardest setting overall. Predicting a new drug is hard, and predicting a new cell line is also hard

because the baseline state changes how the same drug effect moves through the system. The graph gives a stable prior structure, but it cannot remove the difficulty caused by a new cell background. DeepCOP shows a different pattern between these two test settings. Its cold cell result is better than its cold drug result. This should not be explained by the matched control profile alone, because all three models receive that same input. A more likely explanation is that the cold drug split asks the model to generalize drug effects to compounds that were not seen during training, and this seems harder for a feed forward baseline without an explicit graph prior. This helps explain why CAGNN is stronger on PCC than DeepCOP. Drug effects do not appear independently across genes. They spread through connected regulators and pathways. A feed forward network has to infer these relations only from the benchmark samples, while CAGNN already uses known links between genes. The metric comparison provides another insight. Sign accuracy gives a more mixed ranking than PCC and top gene precision. Predicting the correct direction of change is not the same as predicting the full response profile. A model can get many signs right while still missing the relative size or the detailed shape of the response. For this task, PCC and top gene precision give a more useful view of practical prediction quality than sign agreement alone.

The ablation analysis supports the importance of baseline state modeling. Replacing cell identity with control derived cell context improves both MSE and PCC in all three splits, and the gain is much larger in cold cell. This suggests that the measured control profile becomes especially valuable when the test cell line was never seen during training.

The uncertainty analysis answers a different question from the main comparison. It does not ask which model wins the three split comparison. Instead, it asks whether the uncertainty output of CAGNN produces a useful reliability signal. Across all test settings, higher predicted uncertainty is usually associated with larger absolute error, and this relationship is strongest in the hardest cases. The prediction band is shown as $\mu \pm 2\sigma$. Under a Gaussian view, this can be read as a rough 95% interval. In the displayed examples, most shown values fall inside these bands, and the bands are usually wider in the harder cases. This looks reasonable, but it is still only a limited check. A more complete evaluation would measure coverage across all samples and genes.

The warm split data efficiency analysis answers another separate question. It examines whether the graph based model stays useful when the training budget is reduced. CAGNN remains strong when the amount of training data is reduced, which suggests that the biological graph helps when data are limited. In this kind of biology work, repeated measurements are expensive, and many drug cell combinations still have only limited data. A model that stays useful with less data can be more useful than a model that performs well only when data are abundant.

The case studies give a detailed but limited view of attention. When a prediction is accurate, the model often gives high weights to a smaller and easier to read set of graph edges, and some of these edges are consistent with known drug targets or later response genes. When a prediction is weak, large edge weights can still appear, but they are harder to summarize in a useful way. This means attention can be useful as a model clue, but not as direct biological proof. A high attention edge does not prove that the interaction caused the response in the cell.

6.3 Limitations and Future Work

A major limitation of the model is that the biological graph is fixed, while real cell responses change across settings. The same regulatory link may not be equally important in every cell type or every baseline state. Because the graph structure does not change across samples, the model can miss cell specific biology. This is one likely reason why cold cell remains difficult. The framework captures general biological structure, but it does not adapt the graph itself to different cell conditions.

Another limitation comes from the input information. The model uses known drug targets because they are simple and biologically meaningful. But drug target annotations are incomplete, and some compounds act through secondary or poorly characterized effects. In such cases, even a well designed graph model may receive an incomplete starting signal. Some prediction failures may come from limited prior knowledge, not only from model weakness.

The study is also limited by the scope of the data. The main experiments use the 978 landmark genes in LINCS L1000. This keeps the setup manageable and close to the assay design, but it also means that large parts of the transcriptome are not modeled directly. The landmark setting is a practical compromise. It reduces computational cost, but it removes many genes that could help explain longer range responses. Some missing information may be partly reconstructed through correlated landmark patterns, but this remains an indirect view of the full transcriptome. In addition, the dataset contains a limited set of cell lines, many of which are cancer related. The conclusions are strongest for this benchmark setting and should not be taken as direct evidence that the same performance will hold in all biological systems. Extending the analysis to a larger gene set, or to an external transcriptome dataset, would help test how much of the present gain depends on the landmark setting itself.

Another limitation is explanation. High attention weights do not show how biology works by themselves. They only show which graph edges the model used more strongly when making a prediction. For this reason, the attention results should be treated as clues for future lab experiments, not as proof of how a drug works. This is especially important when the prediction itself is weak, because strong weights can still appear in cases that are difficult to explain.

The uncertainty output has limitations as well. CAGNN predicts a Gaussian variance term for each gene, and this is useful for ranking harder and easier cases. However, good ranking does not automatically mean that the uncertainty is fully correct. A well calibrated model should not only assign larger uncertainty to harder cases. It should also produce prediction intervals with the expected coverage. The present thesis shows useful visual patterns, but it does not yet establish full uncertainty calibration.

These limitations suggest several directions for future work. One direction would be to let the graph change with the sample, so that the connections can depend on the specific cell line or baseline state. Another direction would be to improve the drug representation, for example by combining target annotations with other information when targets are incomplete. Better uncertainty calibration would also be useful, so that the uncertainty values are easier to use in later decisions.

A further direction is stronger biological validation. The current case studies show that some high weight edges look reasonable and some do not. A next step would be to compare these edges in a more systematic way with known pathways or with later experiments. That would help separate cases where the model is using biologically meaningful structure from cases where it is only using broad response signals.

6.4 Ethical Considerations

This work uses public perturbation transcriptomics data and does not involve direct experiments on human participants or animals. Even so, several ethical issues remain relevant. Predictions of drug response can appear more certain and more general than they really are. For that reason, the model should not be used as a clinical decision system or as proof that a drug will be effective in patients. The present results come from cell line benchmark data, and they should be interpreted within that scope.

Another ethical issue concerns bias in the data. The LINCS collection contains a limited set of cell lines and experimental conditions. Many cell lines are cancer related, and some biological contexts are represented much better than others. A model trained on such data may perform unevenly across biological settings that are not well covered by the benchmark. This limitation should be acknowledged when presenting the method or discussing any possible downstream use.

Interpretation raises another ethical issue. Attention maps and graph edges can look persuasive, which may encourage users to read them as proof of biological mechanism. That would be misleading. The model can provide hypotheses and useful clues, but these outputs still need external validation. This limitation should be stated clearly to avoid overclaiming biological insight.

6.5 Usage of Generative AI

Generative AI tools were used as writing and coding support during the thesis work. They helped with English revision, text clarity, feedback summary, and some code level tasks such as script drafting and result checking. They were not used to generate the scientific conclusions independently. All methodological choices, experiments, result interpretation, and final text selection were reviewed and decided by the author. Numerical results and figures were produced from the project code and underlying data, and they were checked within the thesis workflow before inclusion. Generative AI supported productivity and language revision, but it did not replace scientific judgment or responsibility for the final content.

7

Conclusion

This thesis examined whether graph neural networks with known biological information can improve prediction of gene expression changes after drug treatment. The answers to the main research questions are summarized below.

7.1 Research Questions

Which graph neural network architecture was most appropriate for this task?

Among the evaluated graph based models, the proposed CAGNN was the most suitable design for this task. It combines a graph attention network with the OmniPath interaction graph, drug target input, matched control expression, and control derived cell context. In the main comparison, it gave higher PCC and top gene precision than the other models in all three splits. It also gave lower MSE than GSNN in all three splits and lower MSE than DeepCOP in warm and cold drug. This does not prove that a GAT based model is always the best choice for every drug response prediction problem. However, within the scope of this thesis, it performed better than the other tested graph based model on the reported metrics.

How accurately can the model generalize to unseen drugs and unseen cell lines?

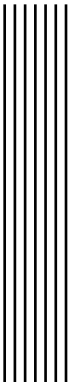
The model worked reasonably well on unseen drugs, but cold cell remained the hardest setting on PCC and top gene precision. In the cold drug split, CAGNN still gave the highest PCC and top gene precision, together with the lowest MSE. This shows that the model can transfer useful information to drugs not seen during training. In the cold cell split, the performance of all methods dropped. DeepCOP gave the lowest MSE there, while CAGNN remained stronger on PCC and top gene precision. This means that a new cell background is harder than a new drug. The uncertainty results support the same conclusion, since cold cell also showed the clearest link between predicted uncertainty and prediction error.

How much does a biologically informed GNN improve over models that do not use network structure?

Compared with DeepCOP, the non graph baseline, CAGNN gave higher PCC in all three splits and lower MSE in warm and cold drug, while DeepCOP was slightly better on cold cell MSE. Compared with GSNN, the graph based baseline, CAGNN gave higher PCC, higher top gene precision, and lower MSE in all three splits. The strongest evidence comes from the consistent gains on PCC and top gene precision across all three splits. This means that the gain was not limited to one evaluation view. The model not only reduced numerical error in most settings, but also preserved the overall response pattern and recovered the strongest changed genes more accurately.

7.2 Conclusion

This thesis shows that known biological information can improve prediction of gene expression changes after drug treatment when it is built into the model in a clear way. In this work, that information came from a molecular interaction graph, known drug targets, and the measured control profile of the sample. When these sources were combined in CAGNN, the model gave higher PCC and top gene precision across all three splits, and lower MSE in warm and cold drug. Cold cell remained the hardest setting on PCC and top gene precision. There is still room for improvement, especially for unseen cell lines, uncertainty calibration, and biological validation.



Bibliography

- [1] A. Subramanian, R. Narayan, and et al., “A gene expression signature-based approach to identify novel mechanisms of action for drugs and chemical compounds,” *Nature Biotechnology*, vol. 35, pp. 463–472, 2017.
- [2] Congressional Budget Office, “Research and development in the pharmaceutical industry,” Tech. Rep. 57025, Congressional Budget Office, Apr. 2021.
- [3] D. Brown, R. Dickman, and A. Schlessinger, “Rethinking target-based drug discovery: Challenges, innovations and the next s-curve,” *Technology Networks*, Dec. 2025. Accessed: 2026-03-14.
- [4] Annual Reviews, “Network-based approaches for drug target identification,” *Annual Review of Biomedical Data Science*, vol. 8, 2025. Discusses the challenges in drug target identification and the complexity of human diseases.
- [5] Drug Hunter, “An unexpected muscle toxicity from az’s covalent program.” <https://drughunter.com/commentary/an-unexpected-muscle-toxicity-from-az-s-covalent-program>, July 2025. Accessed: 2026-03-14.
- [6] N. C. Santos, J. Figueira-Coelho, J. Martins-Silva, and C. Saldanha, “Multidisciplinary utilization of dimethyl sulfoxide: pharmacological, cellular, and molecular aspects,” *Biochemical Pharmacology*, vol. 65, no. 7, pp. 1035–1041, 2003.
- [7] D. Türei, T. Korcsmáros, and J. Saez Rodríguez, “Omnipath: guidelines and gateway for literature curated signaling pathway resources,” *Nature Methods*, vol. 13, no. 12, pp. 966–967, 2016.
- [8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [9] S. Li, H. Hua, and S. Chen, “Graph neural networks for single-cell omics data: a review of approaches and applications,” *Briefings in Bioinformatics*, vol. 26, no. 2, 2025.
- [10] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Moncecchi, “The graph neural network model,” *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [11] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017.

- [12] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, vol. 30, pp. 1024–1034, 2017.
- [13] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform bad for graph representation?," in *Advances in Neural Information Processing Systems*, 2021.
- [14] L. Rampásek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [15] P. Roohani, Y. Wang, Y. Wang, Y. Li, X. Gao, and Y. Chen, "Gears: Gene expression analysis with regression and structure," *Nature Communications*, vol. 14, p. 1234, 2023.
- [16] M. Shahir, J. Smith, A. Lee, and W. Zhang, "Cellograph: Graph neural networks for single-cell perturbation prediction," *Bioinformatics*, vol. 40, no. 2, pp. 456–463, 2024.
- [17] N. J. Evans, G. B. Mills, G. Wu, X. Song, and S. McWeeney, "Graph structured neural networks for perturbation biology," *bioRxiv*, 2024.
- [18] G. Woo *et al.*, "Deepcop: deep learning-based approach to predict gene regulating effects of small molecules," *Bioinformatics*, vol. 36, no. 2, pp. 464–471, 2020.
- [19] C. P. Roca *et al.*, "Regulation flow analysis discovers molecular mechanisms of action from large knowledge databases," *bioRxiv*, 2025.
- [20] Y. Wang *et al.*, "Graphgrp: A graph neural network for predicting drug response based on gene regulatory pathways," *Bioinformatics*, vol. 38, no. 12, pp. 3127–3134, 2022.
- [21] S. Kim, S. Bae, Y. Piao, and K. Jo, "Graph convolutional network for drug response prediction using gene expression data," *Mathematics*, vol. 9, p. 772, 2021.
- [22] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.
- [24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.
- [26] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [27] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?," in *Advances in Neural Information Processing Systems*, pp. 5574–5584, 2017.
- [28] Y. Guo, H. Zhang, H. Hu, J. Wu, J. Cao, C. Y. Hsieh, and B. Yang, "Modelling drug induced cellular perturbation responses with a biologically informed dual branch transformer," *Nature Machine Intelligence*, vol. 8, pp. 96–112, 2026.
- [29] D. Rogers and M. Hahn, "Extended-connectivity fingerprints," *Journal of chemical information and modeling*, vol. 50, no. 5, pp. 742–754, 2010.

- [30] M. Ashburner, C. A. Ball, J. A. Blake, D. Botstein, H. Butler, J. M. Cherry, A. P. Davis, K. Dolinski, S. S. Dwight, J. T. Eppig, *et al.*, "Gene ontology: tool for the unification of biology," *Nature genetics*, vol. 25, no. 1, pp. 25–29, 2000.
- [31] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1735–1742, 2006.
- [32] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 815–823, 2015.
- [33] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the gdpr," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2017.